



TRƯỜNG ĐẠI HỌC QUY NHƠN

TIỂU LUẬN HỌC PHẦN
LÝ THUYẾT TỐI ƯU

THUẬT TOÁN DI TRUYỀN
CƠ SỞ LÝ THUYẾT
VÀ CÁC ỨNG DỤNG

Trình độ đào tạo:	Thạc sĩ
Học viên thực hiện:	Nguyễn Hồ Bảo Thiên
Lớp:	Khoa học dữ liệu K28B
Giảng viên hướng dẫn:	TS. Nguyễn Văn Vũ

Gia Lai, 2026

Mục lục

Danh sách hình vẽ	3
Danh sách bảng	4
Danh mục thuật ngữ	5
Mở đầu	7
1 Cơ sở lý thuyết về Thuật toán Di truyền	8
1.1 Tổng quan về Bài toán Tối ưu và Họ Thuật toán Tiến hóa	8
1.2 Cơ sở Khoa học và Nguyên lý Hoạt động của GA	10
1.3 Các Thành phần và Phép toán Cốt lõi	12
1.4 Quy trình Thuật toán và Điều kiện Dừng	19
1.5 Lập trình Di truyền	22
2 Thực nghiệm	27
2.1 Tối ưu không ràng buộc	27
2.2 Tối ưu có ràng buộc	34
2.3 Bài toán Người du lịch: GA hoán vị kết hợp Tìm kiếm Cục bộ	42
2.4 Hồi quy Symbolic bằng Lập trình Di truyền	45
Kết luận	49
Tài liệu tham khảo	51

Danh sách hình vẽ

1.1	Ví dụ lai ghép trong GA trên hai cách mã hóa. Trên: mã hóa nhị phân với lai ghép đơn điểm. Dưới: mã hóa số thực với lai ghép trộn theo (1.4), $\alpha = 0.75$	15
1.2	Ví dụ lai ghép khớp từng phần trên mã hóa hoán vị của 8 phần tử. Đoạn hoán đổi trực tiếp giữa hai cha mẹ (vị trí 3 đến 5); các vị trí còn lại của cá thể con, giữ nguyên hoặc được sửa theo ánh xạ để tránh trùng lặp. .	16
1.3	Ví dụ lai ghép theo thứ tự trên mã hóa hoán vị của 8 phần tử. Đoạn giữ nguyên từ cha mẹ tương ứng (vị trí 3 đến 5); các vị trí còn lại của cá thể con, điền theo thứ tự xuất hiện trong cha mẹ kia (bỏ qua phần tử đã dùng).	17
1.4	Ví dụ lai ghép theo chu trình trên mã hóa hoán vị của 8 phần tử, với ba chu trình được xác định giữa vị trí phần tử của hai cha mẹ. Con 1 lấy chu trình 1 và 3 từ Cha mẹ 1, chu trình 2 từ Cha mẹ 2; Con 2 lấy ngược lại.	17
1.5	Ví dụ đột biến trong GA trên hai cách mã hóa. Trên: mã hóa nhị phân với đột biến đảo bit tại vị trí thứ 3. Dưới: mã hóa số thực với đột biến Gauss, cộng nhiễu ngẫu nhiên vào vị trí thứ 2 theo (1.5).	18
1.6	Chu trình tiến hóa của Thuật toán Di truyền	20
1.7	Lai ghép trao đổi cây con giữa hai cây cha mẹ (a)–(b), tạo ra hai cây con (c)–(d).	25
1.8	Ba toán tử đột biến minh họa trên cây $x + 3$ (đột biến rút gọn minh họa trên cây lớn hơn $(x + 3) \times x$): (a)–(b) đột biến thay thế cây con, (c)–(d) đột biến điểm, (e)–(f) đột biến rút gọn.	26
2.1	Hàm Easom: đồ thị trong không gian ba chiều và diễn biến giá trị tốt nhất qua các thế hệ. Đường nét đứt đánh dấu thế hệ mà GA chót được nghiệm tốt nhất.	28
2.2	Hàm Rastrigin: mạng lưới cực tiểu cục bộ dày đặc quanh cực tiểu toàn cục, và diễn biến giá trị tốt nhất qua các thế hệ.	29

2.3	Hàm Rosenbrock: thung lũng hẹp và cong, và diễn biến giá trị tốt nhất qua các thế hệ — GA chốt kỷ lục ngay ở thế hệ 11 rồi không cải thiện thêm trong 39 thế hệ còn lại.	31
2.4	Hàm Ackley: phễu lớn ở thang toàn cục phủ gọn sóng cực tiểu cục bộ ở thang nhỏ, và diễn biến giá trị tốt nhất qua các thế hệ.	32
2.5	Hàm Schwefel: cực tiểu toàn cục nằm gần biên miền tìm kiếm, xa vùng trông có vẻ tốt quanh gốc tọa độ, và diễn biến giá trị tốt nhất qua các thế hệ.	33
2.6	Đường hội tụ của GA trên bài toán g01	36
2.7	Đường hội tụ của GA trên bài toán g06	37
2.8	Đường hội tụ của GA trên bài toán g08	38
2.9	Đường hội tụ của GA trên bài toán g11	39
2.10	Đường hội tụ của GA trên bài toán g15	41
2.11	Minh họa một bước 2-opt trên bốn địa điểm: hai cạnh cắt chéo nhau (trái) được thay bằng hai cạnh ngắn hơn, không cắt chéo (phải), bằng cách loại bỏ cạnh $A-D$ và $C-B$ rồi đảo ngược đoạn đường nằm giữa chúng để tạo tour mới $A-C-D-B-A$	43
2.12	Đường hội tụ của GA trên bộ dữ liệu berlin52	44
2.13	Đường hội tụ của GA trên bộ dữ liệu rd100	44
2.14	Đường hội tụ của GA trên bộ dữ liệu ch150	44
2.15	Đường hội tụ của RMSE và MAE qua các thế hệ khi GP dự đoán T_{dew}	46
2.16	Cây biểu thức GP tìm được cho T_{dew} (đã rút gọn theo Phương trình (2.15))	47

Danh sách bảng

1.1	Bảng đối chiếu thuật ngữ Sinh học và Thuật toán Di truyền	11
2.1	Kết quả GA trên năm hàm benchmark không ràng buộc	34
2.2	Kết quả trên bài toán g01	36
2.3	Kết quả trên bài toán g06	37
2.4	Kết quả trên bài toán g08	38
2.5	Kết quả trên bài toán g11	39
2.6	Kết quả trên bài toán g15	40
2.7	Tổng hợp kết quả GA trên năm bài toán chuẩn CEC 2006	41
2.8	Kết quả GA kết hợp 2-opt trên ba bộ dữ liệu TSPLIB95	43
2.9	Năm dòng đầu tiên của dữ liệu thời tiết sau khi lọc còn 4 cột	45

Danh mục thuật ngữ

Bảng dưới đây đối chiếu các thuật ngữ tiếng Anh được sử dụng trong tiểu luận với thuật ngữ tiếng Việt tương ứng. Dạng viết tắt, nếu có, được ghi sau dấu gạch ngang. Trong phần nội dung, các thuật ngữ chỉ xuất hiện ở dạng tiếng Việt hoặc ở dạng viết tắt đã liệt kê tại đây.

Thuật ngữ tiếng Anh	Thuật ngữ tiếng Việt
Benchmark Function	Hàm chuẩn dùng để đánh giá
Binary Encoding	Mã hóa nhị phân
Bit-flip Mutation	Đột biến đảo bit
Blend Crossover — BLX- α	Lai ghép trộn
Bloat	Hiện tượng cây biểu thức phình to
Chromosome	Nhiễm sắc thể
Cycle Crossover — CX	Lai ghép theo chu trình
Elitism	Chiến lược giữ lại cá thể ưu tú
Evolution Strategies — ES	Chiến lược Tiến hóa
Evolutionary Algorithms — EA	Họ Thuật toán Tiến hóa
Evolutionary Programming — EP	Lập trình Tiến hóa
Feasible Region	Miền khả thi
Fitness Function	Hàm độ thích nghi
Gaussian Mutation	Đột biến Gauss
Gene	Gen
Generation	Thế hệ
Genetic Algorithm — GA	Thuật toán Di truyền
Genetic Programming — GP	Lập trình Di truyền
Global Optimum	Cực trị toàn cục
Gradient	Vector đạo hàm riêng
Hoist Mutation	Đột biến rút gọn
Individual	Cá thể
Insertion Mutation	Đột biến chèn
Internal Node	Nút trung gian
Inversion Mutation	Đột biến đảo ngược

Thuật ngữ tiếng Anh	Thuật ngữ tiếng Việt
Local Optimum	Cực trị cục bộ
Local Search	Tìm kiếm cục bộ
Mean Absolute Error — MAE	Sai số tuyệt đối trung bình
Mean Squared Error — MSE	Sai số bình phương trung bình
Memetic Algorithm	Thuật toán tiến hóa kết hợp tìm kiếm cục bộ
Multi-point Crossover	Lai ghép đa điểm
Objective Function	Hàm mục tiêu
Order Crossover — OX	Lai ghép theo thứ tự
Parsimony Pressure	Số hạng phạt theo kích thước cây
Partially Matched Crossover — PMX	Lai ghép khớp từng phần
Permutation Encoding	Mã hóa hoán vị
Point Mutation	Đột biến điểm
Population	Quần thể
Population-based Search	Tìm kiếm dựa trên quần thể
Rank-based Selection	Chọn lọc theo thứ hạng
Real-value Encoding	Mã hóa số thực
Root Mean Squared Error — RMSE	Căn bậc hai của sai số bình phương trung bình
Roulette Wheel Selection	Chọn lọc theo vòng quay Roulette
Search Space	Không gian tìm kiếm
Single-point Crossover	Lai ghép đơn điểm
Stagnation	Quần thể ngừng cải thiện
Subtree Crossover	Lai ghép trao đổi cây con
Subtree Mutation	Đột biến thay thế cây con
Survival of the Fittest	Chọn lọc tự nhiên
Swap Mutation	Đột biến hoán đổi
Tournament Selection	Chọn lọc giải đấu
Travelling Salesman Problem — TSP	Bài toán người du lịch
Tree/Graph Encoding	Mã hóa dựa trên cấu trúc cây hoặc đồ thị
Uniform Crossover	Lai ghép đồng nhất

Mở đầu

Đặt vấn đề

Tối ưu hóa là bài toán trung tâm của nhiều lĩnh vực khoa học kỹ thuật, với mục tiêu tìm ra phương án tốt nhất trong một không gian khả thi theo một tiêu chí đánh giá cho trước. Các phương pháp tối ưu cổ điển dựa trên đạo hàm, dù có cơ sở lý thuyết chặt chẽ và hội tụ nhanh, lại bộc lộ hai hạn chế cố hữu khi đối mặt với các bài toán thực tế: dễ mắc kẹt tại cực trị cục bộ trên hàm mục tiêu không lồi, và đòi hỏi hàm mục tiêu phải khả vi – điều kiện không phải lúc nào cũng thỏa mãn, đặc biệt với các bài toán tổ hợp như định tuyến hay lập lịch. Những hạn chế này thúc đẩy sự phát triển của nhóm phương pháp metaheuristic dựa trên quần thể, trong đó Thuật toán Di truyền (Genetic Algorithm – GA) là đại diện tiêu biểu nhất, mô phỏng nguyên lý chọn lọc tự nhiên của Darwin để tìm kiếm nghiệm mà không cần thông tin đạo hàm hay giả định về tính lồi của hàm mục tiêu.

Mục tiêu

Tiểu luận đặt ra hai mục tiêu chính. Một là hệ thống hóa cơ sở lý thuyết của GA – từ nguồn gốc sinh học, các cách mã hóa nghiệm, đến các thành phần và toán tử cốt lõi cấu thành một chu trình tiến hóa hoàn chỉnh – cùng với Lập trình Di truyền (Genetic Programming – GP), một mở rộng của GA cho phép tiến hóa trực tiếp cấu trúc chương trình hoặc biểu thức thay vì chỉ tối ưu tham số số. Hai là minh họa khả năng áp dụng của hai phương pháp này qua các thực nghiệm cụ thể, đối chiếu trực tiếp với các mốc nghiệm tối ưu hoặc quan hệ vật lý đã biết, nhằm đánh giá khách quan chất lượng nghiệm mà GA/GP tìm được.

Chương 1

Cơ sở lý thuyết về Thuật toán Di truyền

Chương này trình bày các nền tảng lý thuyết cần thiết của Thuật toán Di truyền. Nội dung chương được tổ chức thành năm phần chính: mục đầu tiên phát biểu bài toán tối ưu hóa tổng quát và giới thiệu họ thuật toán tiến hóa; mục thứ hai trình bày cơ sở sinh học cũng như nguyên lý hoạt động của GA; mục thứ ba phân tích chi tiết các thành phần và toán tử cốt lõi; mục thứ tư tổng hợp quy trình tổng thể của thuật toán và điều kiện dừng. Phần cuối trình bày Lập trình Di truyền (Genetic Programming – GP), một phương pháp phát triển từ các nguyên lý của Thuật toán Di truyền (GA), cho phép tiến hóa trực tiếp cấu trúc của chương trình hoặc biểu thức để tìm ra lời giải phù hợp.

1.1 Tổng quan về Bài toán Tối ưu và Họ Thuật toán Tiến hóa

Bài toán Tối ưu hóa và Không gian Tìm kiếm

Bài toán tối ưu hóa tổng quát có thể được phát biểu như sau: tìm một vector biến quyết định $x = (x_1, x_2, \dots, x_n)$ sao cho hàm mục tiêu $f(x)$ đạt giá trị cực tiểu hoặc cực đại, trong khi x vẫn thỏa mãn một tập ràng buộc xác định miền khả thi Ω :

$$\min_{x \in \Omega} f(x) \quad \text{hoặc} \quad \max_{x \in \Omega} f(x). \quad (1.1)$$

Miền $\Omega \subseteq \mathbb{R}^n$, còn được gọi là không gian tìm kiếm, thường được xác định bởi một hệ ràng buộc đẳng thức và bất đẳng thức trên các biến quyết định. Bài toán được phân loại là **có ràng buộc** khi Ω là một tập con thực sự của không gian biến, và **không ràng buộc** khi Ω trùng với toàn bộ không gian biến.

Trong thực tiễn, tối ưu hóa thường đối mặt với hai thách thức chủ yếu như sau.

- **Vấn đề cực trị cục bộ.** Khi hàm mục tiêu không lồi hoặc đa cực trị, các thuật toán chỉ khai thác thông tin cục bộ quanh điểm hiện tại, chẳng hạn đạo hàm, có xu hướng hội tụ về cực trị cục bộ gần với điểm khởi tạo thay vì cực trị toàn cục trên toàn miền Ω . Đây chính là hạn chế cốt lõi của các phương pháp tối ưu dựa trên gradient khi áp dụng cho bài toán không lồi.
- **Độ phức tạp tổ hợp của bài toán NP-hard.** Đối với nhiều lớp bài toán tối ưu tổ hợp, tiêu biểu như bài toán người du lịch hay bài toán lập lịch, số lượng phương án khả thi tăng theo hàm mũ hoặc giai thừa của kích thước bài toán. Kết quả là, việc duyệt toàn bộ không gian tìm kiếm trở nên bất khả thi về mặt tính toán ngay cả với các bài toán có kích thước vừa phải.

Hai thách thức nêu trên chính là động lực thúc đẩy sự phát triển của các phương pháp tìm kiếm không dựa trên đạo hàm, đồng thời có khả năng khám phá nhiều vùng khác nhau của không gian tìm kiếm. Một trong những hướng tiếp cận tiêu biểu của nhóm phương pháp này là họ thuật toán tiến hóa, được giới thiệu trong mục tiếp theo.

Họ Thuật toán Tiến hóa

Các phương pháp tối ưu cổ điển dựa trên đạo hàm như Gradient Descent hay phương pháp Newton vận hành trên một điểm duy nhất: ở mỗi bước lặp, điểm hiện tại được dịch chuyển theo hướng mà đạo hàm chỉ ra. Cách làm này hội tụ nhanh và có cơ sở lý thuyết chặt chẽ, nhưng đòi hỏi hàm mục tiêu phải khả vi và chỉ bảo đảm tìm được cực trị cục bộ nằm gần điểm khởi tạo.

Ở một hướng tiếp cận khác, **metaheuristic** là các chiến lược tìm kiếm ở mức tổng quát, được xây dựng để làm việc trong chính những điều kiện mà phương pháp cổ điển không áp dụng được: hàm mục tiêu không khả vi, có nhiều, không lồi, hoặc thậm chí không tồn tại biểu thức giải tích tường minh. Sự đánh đổi ở đây là rõ ràng. Metaheuristic từ bỏ bảo đảm về tính tối ưu tuyệt đối để nhận lại phạm vi áp dụng rộng hơn nhiều, và mục tiêu thực tế của nó là đạt được một nghiệm đủ tốt trong khoảng thời gian chấp nhận được.

Trong số các metaheuristic, nhóm **tìm kiếm dựa trên quần thể** duy trì đồng thời nhiều nghiệm ứng viên thay vì một nghiệm duy nhất, rồi để cả tập hợp đó cùng tiến hóa qua các thế hệ. Việc khảo sát nhiều vùng của không gian tìm kiếm cùng lúc làm giảm đáng kể nguy cơ hội tụ sớm về một cực trị cục bộ. Quan trọng hơn, các nghiệm trong quần thể có thể trao đổi thông tin với nhau để tạo ra nghiệm mới, điều mà một thuật toán chỉ theo dõi một điểm không thể thực hiện được. **Họ Thuật toán Tiến hóa** là đại diện tiêu biểu nhất của nhóm này, với quá trình tìm kiếm được dẫn dắt bởi nguyên lý chọn lọc tự nhiên của Darwin.

Mọi thuật toán thuộc họ EA đều vận hành theo một khung chung gồm năm bước: khởi tạo ngẫu nhiên một quần thể nghiệm, đánh giá chất lượng của từng nghiệm, ưu tiên những nghiệm tốt hơn khi chọn cá thể sinh sản, sinh nghiệm mới bằng các toán tử biến đổi ngẫu nhiên, và lặp lại chu trình cho tới khi thỏa mãn điều kiện dừng. Bốn nhánh chính của họ này khác nhau chủ yếu ở cách biểu diễn nghiệm và ở toán tử biến đổi được đặt làm trọng tâm:

- **Thuật toán Di truyền:** đặt lai ghép làm toán tử tìm kiếm chính, kết hợp thông tin di truyền từ hai cá thể cha mẹ để tạo ra cá thể con. Đây là đối tượng nghiên cứu chính của tiểu luận.
- **Chiến lược Tiến hóa:** do Rechenberg và Schwefel phát triển cho bài toán tối ưu tham số thực, lấy đột biến Gauss làm toán tử chính và bổ sung cơ chế tự thích nghi biên độ đột biến trong quá trình tiến hóa.
- **Lập trình Tiến hóa:** do Fogel đề xuất, tiến hóa chủ yếu bằng đột biến trên biểu diễn hành vi của cá thể và hầu như không sử dụng lai ghép.
- **Lập trình Di truyền:** do Koza phát triển, mở rộng GA để tiến hóa trực tiếp các cấu trúc dạng cây biểu thức hoặc chương trình máy tính, thường dùng để tự động khám phá công thức toán học hoặc chương trình cho một tác vụ đặt ra từ trước.

Trong phạm vi tiểu luận này, Thuật toán Di truyền được khảo sát trên cả ba cách mã hóa nghiệm. Mã hóa số thực được dùng cho bài toán tối ưu không ràng buộc và bài toán tối ưu có ràng buộc, mã hóa hoán vị cho bài toán người du lịch, còn mã hóa dạng cây biểu thức theo nguyên lý Lập trình Di truyền được dùng cho bài toán hồi quy symbolic. Khung vận hành năm bước vừa nêu sẽ được cụ thể hóa dần ở các mục tiếp theo, lần lượt qua cách mã hóa nghiệm, hàm độ thích nghi, và ba nhóm toán tử chọn lọc, lai ghép và đột biến; riêng nhánh Lập trình Di truyền được trình bày tách riêng ở Mục 1.5.

1.2 Cơ sở Khoa học và Nguyên lý Hoạt động của GA

Nguồn gốc Lịch sử và Nguyên lý Sinh học

Thuật toán Di truyền do John Holland đề xuất năm 1975 tại Đại học Michigan trong công trình *Adaptation in Natural and Artificial Systems* [1], và sau đó được David Goldberg hệ thống hóa cũng như phổ biến rộng rãi hơn thông qua cuốn sách năm 1989 *Genetic Algorithms in Search, Optimization, and Machine Learning* [2].

Ý tưởng nền tảng của GA là mô phỏng ba nguyên lý cốt lõi trong học thuyết tiến hóa của Darwin:

- **Chọn lọc tự nhiên:** những cá thể có khả năng thích nghi tốt hơn với môi trường sẽ có xác suất sống sót và sinh sản cao hơn.
- **Di truyền:** các đặc điểm của cá thể cha mẹ được truyền lại cho thế hệ con thông qua vật chất di truyền.
- **Biến dị:** các sai khác ngẫu nhiên giữa các cá thể, hình thành thông qua lai ghép và đột biến, tạo nên tính đa dạng di truyền – điều kiện tiên quyết để quá trình chọn lọc có tác dụng.

Dựa trên ba nguyên lý nêu trên, GA thiết lập một phép tương ứng giữa các khái niệm sinh học và các thành phần thuật toán. Phép tương ứng này được tóm tắt trong Bảng 1.1.

Bảng 1.1: Bảng đối chiếu thuật ngữ Sinh học và Thuật toán Di truyền

Sinh học	Thuật ngữ tiếng Anh	Thuật toán Di truyền
Nhiễm sắc thể	Chromosome	Mã hóa nghiệm
Gen	Gene	Một biến thành phần trong nghiệm
Cá thể	Individual	Một nghiệm ứng viên
Quần thể	Population	Tập hợp các nghiệm tại một thế hệ
Độ thích nghi	Fitness	Giá trị hàm độ thích nghi
Thế hệ	Generation	Một vòng lặp tiến hóa

Nhờ phép tương ứng này, một bài toán tối ưu bất kỳ có thể được diễn dịch thành một quá trình “tiến hóa nhân tạo”. Cụ thể, thuật toán xuất phát từ một quần thể các nghiệm ngẫu nhiên; trong suốt quá trình tiến hóa, các nghiệm có chất lượng tốt hơn sẽ có nhiều cơ hội được chọn làm cha mẹ hơn. Qua nhiều thế hệ, quần thể dần dịch chuyển về phía những vùng của không gian tìm kiếm có giá trị hàm mục tiêu tốt.

Phương pháp Mã hóa Nghiệm

Việc áp dụng GA cho một bài toán cụ thể bắt đầu bằng bước mã hóa: mỗi nghiệm cần được biểu diễn dưới dạng một “nhiễm sắc thể” mà các toán tử di truyền có thể thao tác trực tiếp. Bốn cách mã hóa phổ biến nhất được trình bày như sau.

- **Mã hóa nhị phân:** nghiệm được biểu diễn bằng một chuỗi bit gồm 0 và 1, chẳng hạn 10110010. Đây là cách mã hóa nguyên thủy nhất của GA cổ điển do Holland đề xuất, phù hợp với các bài toán rời rạc hoặc logic. Tuy nhiên, khi áp dụng cho biến số thực, cách mã hóa này đòi hỏi một bước rời rạc hóa, làm giảm độ chính xác của nghiệm.

- **Mã hóa số thực:** nghiệm được biểu diễn dưới dạng một dãy số thực, chẳng hạn $[3.14, -0.05, 12.8]$, trong đó mỗi phần tử tương ứng trực tiếp với một biến quyết định. Phương pháp mã hóa này đặc biệt phù hợp với các bài toán tối ưu liên tục nhiều biến, đồng thời tránh được sai số do giới hạn độ chính xác khi biểu diễn nghiệm bằng mã nhị phân.
- **Mã hóa hoán vị:** nghiệm là một thứ tự sắp xếp của một tập hợp phần tử, chẳng hạn $[3, 1, 4, 2, 5]$. Đây là cách mã hóa tự nhiên cho các bài toán định tuyến và lập lịch như TSP hay bài toán phân công công việc, nơi nghiệm về bản chất là một hoán vị.
- **Mã hóa dựa trên cấu trúc:** nghiệm được biểu diễn dưới dạng cây biểu thức hoặc đồ thị, chủ yếu được sử dụng trong Lập trình Di truyền nhằm tiến hóa các công thức hoặc chương trình.

Cần lưu ý rằng việc lựa chọn cách mã hóa quyết định trực tiếp đến loại toán tử lai ghép và đột biến có thể áp dụng. Vấn đề này sẽ được phân tích chi tiết trong Mục 1.3.

1.3 Các Thành phần và Phép toán Cốt lõi

Hàm độ thích nghi và Ràng buộc

Hàm độ thích nghi được sử dụng để đánh giá chất lượng của từng cá thể trong quần thể và làm cơ sở cho quá trình chọn lọc. Đối với bài toán không có ràng buộc, hàm độ thích nghi thường được xây dựng dựa trên hàm mục tiêu. Với bài toán cực tiểu hóa, nghiệm tốt là nghiệm có giá trị hàm mục tiêu càng nhỏ nhưng do GA thường ưu tiên các cá thể có độ thích nghi càng lớn, hàm mục tiêu có thể được biến đổi, chẳng hạn bằng cách đổi dấu, để giá trị độ thích nghi tăng tương ứng với mức độ tốt của nghiệm.

Đối với bài toán có ràng buộc, trọng tâm của tiểu luận này, cách tiếp cận kinh điển là đưa ràng buộc vào hàm mục tiêu thông qua một số hạng phạt, tạo thành hàm thích nghi đã phạt:

$$F(x) = f(x) \pm \sum_i \lambda_i \cdot g_i(x), \quad (1.2)$$

trong đó $g_i(x)$ đo mức độ vi phạm ràng buộc thứ i (nhận giá trị 0 nếu ràng buộc được thỏa mãn), và $\lambda_i > 0$ là hệ số phạt tương ứng. Dấu của số hạng phạt cũng như độ lớn của λ_i được chọn tùy theo bài toán là cực tiểu hay cực đại hóa, sao cho một cá thể vi phạm ràng buộc luôn bị đánh giá thấp hơn một cá thể khả thi có cùng giá trị hàm mục tiêu.

Hàm phạt tính tồn tại một hạn chế cố hữu: hệ số λ_i phải được xác định thủ công và giữ cố định trong suốt quá trình tiến hóa. Nếu chọn λ_i quá nhỏ, quần thể có xu hướng bỏ qua các ràng buộc; ngược lại, nếu chọn λ_i quá lớn, số hạng phạt sẽ lấn át

hoàn toàn hàm mục tiêu, khiến quần thể chỉ tập trung thỏa mãn ràng buộc mà không còn khả năng phân biệt giữa các nghiệm khả thi tốt và kém. Để khắc phục hạn chế này, nhiều kỹ thuật xử lý ràng buộc nâng cao đã được đề xuất, không còn dựa vào một hệ số phạt cố định.

Quy tắc khả thi của Deb [3] thay hàm phạt bằng một quy tắc so sánh trực tiếp giữa hai cá thể, dựa trên ba trường hợp: (i) giữa hai cá thể đều khả thi, cá thể có giá trị hàm mục tiêu tốt hơn luôn được ưu tiên; (ii) giữa một cá thể khả thi và một cá thể không khả thi, cá thể khả thi luôn được ưu tiên bất kể giá trị hàm mục tiêu; (iii) giữa hai cá thể đều không khả thi, cá thể có tổng mức vi phạm ràng buộc nhỏ hơn được ưu tiên. Quy tắc này loại bỏ hoàn toàn nhu cầu xác định hệ số phạt, vì việc so sánh không còn cộng gộp mức vi phạm vào hàm mục tiêu.

Ngưỡng ε giảm dần do Takahama và Sakai đề xuất [4] không yêu cầu mức vi phạm bằng 0 ngay từ đầu. Thay vào đó, cá thể được xem là khả thi khi tổng mức vi phạm không vượt quá ngưỡng $\varepsilon(t)$. Ngưỡng này bắt đầu ở giá trị dương, cho phép quần thể khám phá cả vùng không khả thi, rồi giảm dần về 0 tại thế hệ T_c , từ đó hướng quần thể hội tụ vào miền khả thi.

Các Phép chọn lọc

Toán tử chọn lọc quyết định những cá thể trong quần thể hiện tại được lựa chọn làm cha mẹ để tạo ra thế hệ tiếp theo. Về nguyên tắc, các cá thể có độ thích nghi cao sẽ có cơ hội được lựa chọn lớn hơn, qua đó làm tăng khả năng các đặc điểm tốt được duy trì và truyền sang thế hệ sau. Ba phương pháp chọn lọc phổ biến được trình bày dưới đây.

- **Vòng quay Roulette:** Xác suất chọn cá thể i tỉ lệ thuận với độ thích nghi của cá thể đó so với tổng độ thích nghi của toàn bộ quần thể:

$$P_i = \frac{f_i}{\sum_{j=1}^N f_j}. \quad (1.3)$$

Hạn chế của phương pháp này xuất hiện khi một cá thể có độ thích nghi vượt trội so với phần còn lại. Khi đó, cá thể này chiếm phần lớn xác suất được lựa chọn, làm tăng áp lực chọn lọc và có thể khiến tính đa dạng của quần thể giảm nhanh.

- **Giải đấu:** Chọn ngẫu nhiên k cá thể từ quần thể, sau đó chọn cá thể tốt nhất trong nhóm làm cha mẹ. Tham số k , được gọi là kích thước giải đấu, là một yếu tố điều khiển áp lực chọn lọc: k càng lớn thì cá thể tốt càng có lợi thế trong quá trình lựa chọn, quần thể thường hội tụ nhanh hơn, nhưng đồng thời nguy cơ mất đa dạng và hội tụ sớm cũng tăng.

- **Chọn lọc theo thứ hạng:** Các cá thể được sắp xếp theo độ thích nghi và việc lựa chọn được thực hiện dựa trên thứ hạng thay vì giá trị độ thích nghi tuyệt đối. Nhờ đó, sự chênh lệch quá lớn giữa các giá trị độ thích nghi không khiến một cá thể chiếm ưu thế quá mức về xác suất được lựa chọn như trong phương pháp vòng quay Roulette.

Bên cạnh các toán tử chọn lọc cha mẹ nêu trên, chiến lược lựa chọn cá thể chuyển sang thế hệ sau còn bao gồm cơ chế giữ lại cá thể ưu tú, được trình bày riêng trong Mục 1.3 do vai trò và cách áp dụng của nó khác biệt so với chọn lọc cha mẹ thông thường.

Các Phép lai ghép

Toán tử lai ghép kết hợp vật chất di truyền của hai cá thể cha mẹ để tạo ra cá thể con mới. Tần suất lai ghép p_c là xác suất một cặp cha mẹ đã chọn thực sự tham gia lai ghép; phần còn lại được sao chép nguyên vẹn sang thế hệ con. p_c thường được đặt ở mức cao để đa số cặp cha mẹ tham gia lai ghép, giá trị cụ thể tùy vào bài toán và được xác định bằng thực nghiệm.

Đối với mã hóa nhị phân và số thực, ba toán tử lai ghép phổ biến nhất là:

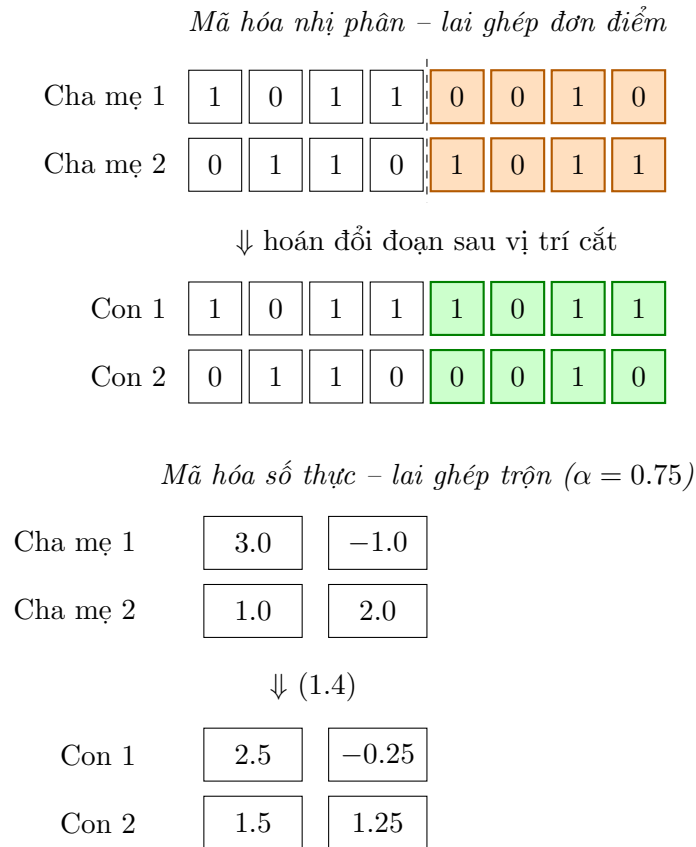
- **Lai ghép đơn điểm:** chọn ngẫu nhiên một vị trí cắt trên chuỗi nhiễm sắc thể. Hai cá thể con được tạo ra bằng cách hoán đổi các đoạn nằm sau vị trí cắt giữa hai cha mẹ.
- **Lai ghép đa điểm:** là dạng tổng quát của lai ghép đơn điểm với nhiều vị trí cắt; các đoạn xen kẽ giữa hai cha mẹ được hoán đổi cho nhau.
- **Lai ghép đồng nhất:** giá trị của mỗi gen tại vị trí con được quyết định độc lập và ngẫu nhiên, lấy từ cha hoặc mẹ, không phụ thuộc vào vị trí của gen trong chuỗi.

Đối với mã hóa số thực, một họ toán tử quan trọng khác là lai ghép trộn, cũng chính là toán tử được sử dụng trong phần cài đặt của tiểu luận. Xét hai cá thể cha mẹ $p_1, p_2 \in \mathbb{R}^n$ và hệ số trộn α được lấy mẫu ngẫu nhiên, có thể nhận giá trị nằm ngoài khoảng $[0, 1]$, hai cá thể con được tạo ra theo tổ hợp tuyến tính:

$$c_1 = \alpha \cdot p_1 + (1 - \alpha) \cdot p_2, \quad c_2 = \alpha \cdot p_2 + (1 - \alpha) \cdot p_1. \quad (1.4)$$

Khi α được lấy mẫu từ một khoảng mở rộng ra ngoài $[0, 1]$, chẳng hạn $\alpha \sim \mathcal{U}(-0.25, 1.25)$, cá thể con có thể nằm ngoài đoạn thẳng nối p_1 và p_2 . Nhờ vậy, toán tử này cân bằng được giữa khả năng khai thác (khai thác vùng lân cận cha mẹ) và khả năng khám phá (tiếp cận các vùng nằm ngoài đoạn nối hai cha mẹ), thay vì chỉ co dần khoảng tìm kiếm qua các thế hệ.

Hình 1.1 minh họa lai ghép đơn điểm trên mã hóa nhị phân và lai ghép trộn trên mã hóa số thực – hai cách mã hóa được nhắc đến nhiều nhất trong chương này, lần lượt là cách mã hóa cổ điển của GA và cách mã hóa dùng trong phần cài đặt thực nghiệm của tiểu luận. Với mã hóa nhị phân, vị trí cắt được chọn ngẫu nhiên sau gen thứ 4; đoạn từ vị trí 5 đến 8 được hoán đổi giữa hai cha mẹ để tạo ra hai cá thể con. Với mã hóa số thực, hai cá thể con được tính trực tiếp theo công thức lai ghép trộn (1.4) với $\alpha = 0.75$.



Hình 1.1: Ví dụ lai ghép trong GA trên hai cách mã hóa. Trên: mã hóa nhị phân với lai ghép đơn điểm. Dưới: mã hóa số thực với lai ghép trộn theo (1.4), $\alpha = 0.75$.

Đối với mã hóa hoán vị trong các bài toán định tuyến và lập lịch, việc hoán đổi trực tiếp từng đoạn như các toán tử trên có thể tạo ra cá thể con vi phạm tính hoán vị do có phần tử bị lặp lại hoặc thiếu sót. Vì vậy, các toán tử lai ghép chuyên biệt cần được sử dụng:

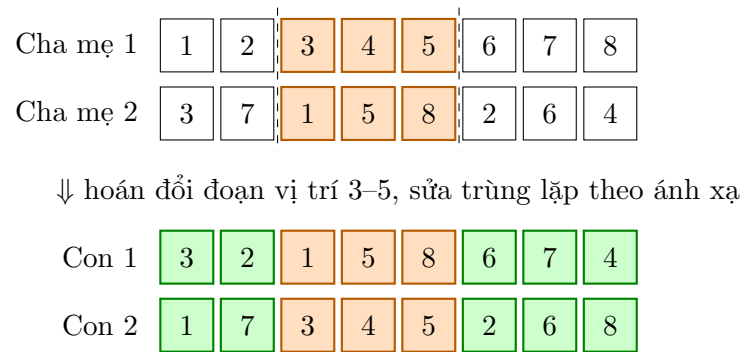
- **Lai ghép khớp từng phần:** hoán đổi một đoạn con giữa hai cha mẹ theo cách tương tự lai ghép đơn điểm, sau đó điều chỉnh các vị trí còn lại dựa trên ánh xạ tương ứng từng phần nhằm đảm bảo tính hoán vị của cá thể con.
- **Lai ghép theo thứ tự:** giữ nguyên một đoạn con từ cha mẹ thứ nhất, các phần tử còn lại được điền vào theo đúng thứ tự xuất hiện của chúng trong cha mẹ thứ hai.

- **Lai ghép theo chu trình:** xác định các “chu trình” giữa vị trí phần tử của hai cha mẹ; mỗi cá thể con được tạo ra bằng cách giữ nguyên các chu trình xen kẽ lấy từ mỗi cha mẹ.

Để tiện so sánh, cả ba ví dụ dưới đây đều xuất phát từ cùng một cặp cha mẹ là hai hoán vị của 8 phần tử.

Hình 1.2 minh họa lai ghép khớp từng phần. Đoạn từ vị trí 3 đến 5 được hoán đổi trực tiếp giữa hai cha mẹ, đồng thời tạo ra một ánh xạ tương ứng từng cặp giá trị trong đoạn đó. Đối với các vị trí còn lại, giá trị gốc của cha mẹ được giữ nguyên nếu không gây trùng lặp với đoạn vừa hoán đổi; nếu trùng lặp, giá trị được thay thế theo ánh xạ đã tạo cho đến khi thu được một giá trị chưa xuất hiện trong cá thể con.

Mã hóa hoán vị – Lai ghép khớp từng phần



Hình 1.2: Ví dụ lai ghép khớp từng phần trên mã hóa hoán vị của 8 phần tử. Đoạn hoán đổi trực tiếp giữa hai cha mẹ (vị trí 3 đến 5); các vị trí còn lại của cá thể con, giữ nguyên hoặc được sửa theo ánh xạ để tránh trùng lặp.

Hình 1.3 minh họa lai ghép theo thứ tự trên cùng cặp cha mẹ – toán tử được sử dụng trong phần cài đặt thực nghiệm cho bài toán người du lịch tại Chương 2. Đoạn từ vị trí 3 đến 5 được giữ nguyên từ cha mẹ tương ứng để tạo thành cá thể con cùng hàng; các vị trí còn lại của cá thể con được điền theo đúng thứ tự xuất hiện của các phần tử chưa dùng trong cha mẹ còn lại.

Mã hóa hoán vị – Lai ghép theo thứ tự

Cha mẹ 1	1	2	3	4	5	6	7	8
Cha mẹ 2	3	7	1	5	8	2	6	4

↓ giữ đoạn vị trí 3–5, điền phần còn lại theo thứ tự ở cha mẹ kia

Con 1	7	1	3	4	5	8	2	6
Con 2	2	3	1	5	8	4	6	7

Hình 1.3: Ví dụ lai ghép theo thứ tự trên mã hóa hoán vị của 8 phần tử. Đoạn giữ nguyên từ cha mẹ tương ứng (vị trí 3 đến 5); các vị trí còn lại của cá thể con, điền theo thứ tự xuất hiện trong cha mẹ kia (bỏ qua phần tử đã dùng).

Hình 1.4 minh họa lai ghép theo chu trình trên cùng cặp cha mẹ. Ba chu trình được xác định giữa vị trí phần tử của hai cha mẹ; mỗi cá thể con được tạo ra bằng cách giữ nguyên các chu trình này nhưng luân phiên lấy giá trị từ mỗi cha mẹ.

Mã hóa hoán vị – Lai ghép theo chu trình

Chu trình 1: cam Chu trình 2: xanh lá Chu trình 3: xanh dương

Cha mẹ 1	1	2	3	4	5	6	7	8
Cha mẹ 2	3	7	1	5	8	2	6	4

↓ luân phiên lấy chu trình từ mỗi cha mẹ

Con 1	1	7	3	4	5	2	6	8
Con 2	3	2	1	5	8	6	7	4

Hình 1.4: Ví dụ lai ghép theo chu trình trên mã hóa hoán vị của 8 phần tử, với ba chu trình được xác định giữa vị trí phần tử của hai cha mẹ. Con 1 lấy chu trình 1 và 3 từ Cha mẹ 1, chu trình 2 từ Cha mẹ 2; Con 2 lấy ngược lại.

Các Phép đột biến

Toán tử đột biến tạo ra biến dị ngẫu nhiên trên một cá thể. Tần suất đột biến p_m trên mỗi gen quyết định mức độ biến dị đưa vào quần thể ở mỗi thế hệ; giá trị cụ thể phụ thuộc vào cách mã hóa và bài toán, được xác định bằng thực nghiệm. Vai trò chính của đột biến là duy trì đa dạng di truyền cho quần thể, đồng thời giúp thuật toán thoát ra khỏi những vùng mà toán tử lai ghép một mình không thể tiếp cận được. Một số dạng đột biến tiêu biểu bao gồm:

- **Đột biến đảo bit:** áp dụng cho mã hóa nhị phân; mỗi bit được đảo giá trị ($0 \leftrightarrow 1$) độc lập với xác suất p_m .

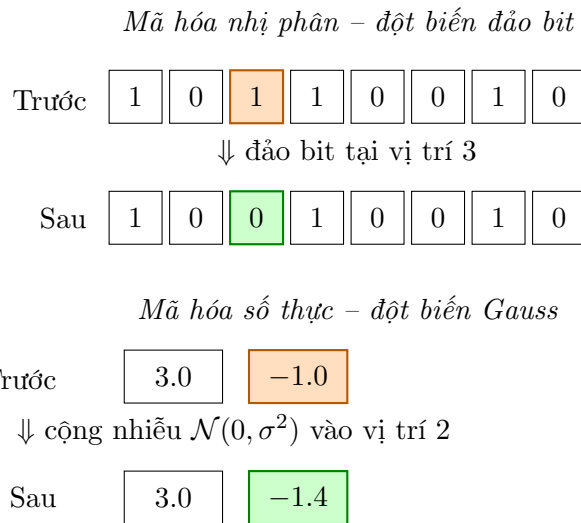
- **Đột biến Gauss:** áp dụng cho mã hóa số thực; mỗi biến x_i được cộng thêm một nhiễu ngẫu nhiên, phổ biến nhất là nhiễu Gauss:

$$x'_i = x_i + \mathcal{N}(0, \sigma_i^2), \quad (1.5)$$

trong đó độ lệch chuẩn σ_i thường được đặt tỉ lệ với độ rộng miền giá trị của biến x_i , nhằm đảm bảo bước đột biến có ý nghĩa tương đương giữa các biến có thang đo khác nhau.

- **Đột biến hoán đổi, đảo ngược và chèn:** áp dụng cho mã hóa hoán vị; lần lượt là hoán đổi vị trí hai phần tử, đảo ngược thứ tự một đoạn con, và di chuyển một phần tử sang vị trí khác trong chuỗi. Cả ba thao tác đều bảo toàn tính hoán vị của cá thể.

Tương tự, Hình 1.5 minh họa đột biến đảo bit trên mã hóa nhị phân và đột biến Gauss trên mã hóa số thực. Với mã hóa nhị phân, gen tại vị trí 3 được đảo giá trị từ 1 thành 0. Với mã hóa số thực, giá trị tại vị trí 2 được cộng thêm một nhiễu Gauss theo (1.5), ở đây lấy mẫu được $\mathcal{N}(0, \sigma^2) = -0.4$, cho ra giá trị mới -1.4 .



Hình 1.5: Ví dụ đột biến trong GA trên hai cách mã hóa. Trên: mã hóa nhị phân với đột biến đảo bit tại vị trí thứ 3. Dưới: mã hóa số thực với đột biến Gauss, cộng nhiễu ngẫu nhiên vào vị trí thứ 2 theo (1.5).

Chiến lược giữ lại Cá thể Ưu tú

Do các quá trình chọn lọc, lai ghép và đột biến có thể làm thay đổi hoặc loại bỏ các cá thể tốt, GA thông thường không đảm bảo cá thể tốt nhất của thế hệ hiện tại được duy trì nguyên vẹn ở thế hệ kế tiếp. Vì vậy, độ thích nghi tốt nhất của quần thể có thể giảm giữa hai thế hệ liên tiếp. **Chiến lược Elitism**, do De Jong đề xuất [5], được sử dụng để khắc phục vấn đề này bằng cách giữ lại k cá thể có độ thích nghi cao nhất của

thế hệ hiện tại và chuyển trực tiếp sang thế hệ kế tiếp mà không thực hiện lai ghép hay đột biến.

Gọi E_t là tập gồm k cá thể ưu tú của thế hệ t . Khi đó, các cá thể này được đưa trực tiếp vào thế hệ $t + 1$:

$$E_t \subseteq P_{t+1}, \quad |E_t| = k.$$

Nhờ đó, nếu bài toán là **cực đại hóa độ thích nghi**, độ thích nghi tốt nhất của quần thể không thể giảm qua các thế hệ:

$$\boxed{f_{\text{best}}(t+1) \geq f_{\text{best}}(t)}$$

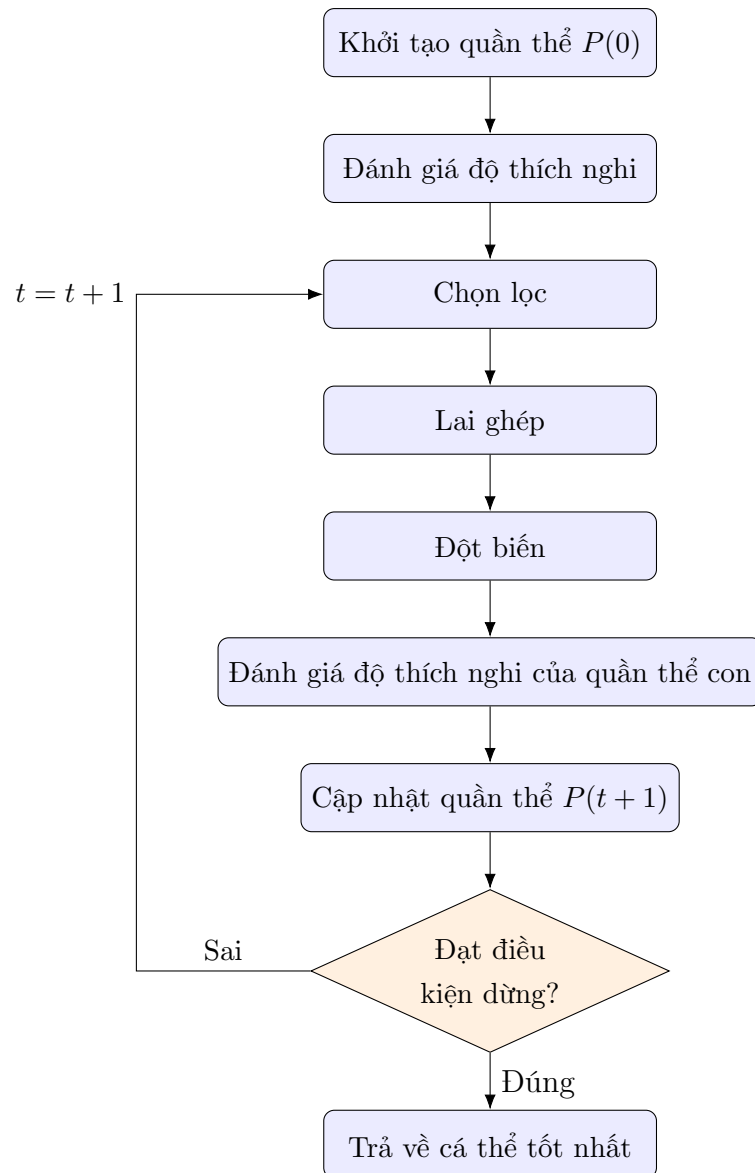
Đây là đặc tính quan trọng nhất của Elitism, đảm bảo rằng một nghiệm tốt đã tìm được sẽ không bị mất do các phép toán tiến hóa ở các thế hệ sau.

Kích thước k cần được lựa chọn phù hợp. Nếu k quá nhỏ, số lượng cá thể được bảo toàn trực tiếp ít, nên tác động của Elitism đối với việc duy trì các nghiệm tốt bị hạn chế. Ngược lại, nếu k quá lớn, một phần lớn quần thể được giữ nguyên qua các thế hệ, làm giảm khả năng khám phá những vùng mới của không gian tìm kiếm, đồng thời có thể làm giảm tính đa dạng và khiến quần thể hội tụ sớm.

1.4 Quy trình Thuật toán và Điều kiện Dừng

Sơ đồ khối và Mã giả

Phối hợp các thành phần đã trình bày trong Mục 1.3, một thế hệ của GA điển hình bao gồm chu trình lặp lại sau: chọn lọc cha mẹ, lai ghép, đột biến, đánh giá quần thể con, rồi cập nhật quần thể mới với cơ chế elitism. Chu trình này được minh họa tổng quát trong Hình 1.6.



Hình 1.6: Chu trình tiến hóa của Thuật toán Di truyền

Để mô tả chính xác hơn, Thuật toán 1 trình bày chu trình tiến hóa trên dưới dạng mã giả (pseudocode).

Thuật toán 1 Mã giả tổng quát của Thuật toán Di truyền

```
1:  $t \leftarrow 0$ 
2: Khởi tạo quần thể ban đầu  $P(0)$  ngẫu nhiên
3: Đánh giá độ thích nghi của  $P(0)$ 
4: while điều kiện dừng chưa thỏa mãn do
5:    $t \leftarrow t + 1$ 
6:   Chọn lọc tập cha mẹ  $S(t)$  từ  $P(t - 1)$ 
7:   Thực hiện lai ghép trên  $S(t)$ , tạo tập con  $C(t)$ 
8:   Thực hiện đột biến trên  $C(t)$ , tạo tập con  $C'(t)$ 
9:   Đánh giá độ thích nghi của  $C'(t)$ 
10:   $P(t) \leftarrow$  tạo quần thể mới từ  $C'(t)$ , giữ lại  $k$  cá thể ưu tú
11: end while
12: return cá thể tốt nhất tìm được
```

Tiêu chuẩn Dừng và Đánh giá sự Hội tụ

Một thuật toán tiến hóa cần có tiêu chí dừng rõ ràng để xác định thời điểm kết thúc quá trình tiến hóa. Trong thực tế, thay vì chỉ sử dụng một tiêu chí duy nhất, nhiều tiêu chí dừng thường được kết hợp để đánh giá cả số lần lặp, mức độ cải thiện của nghiệm và sự hội tụ của quần thể. Ba tiêu chí dừng phổ biến được trình bày dưới đây.

- **Số thế hệ tối đa $G_{\max}$:** đây là tiêu chí đơn giản, đồng thời cho phép kiểm soát trực tiếp số lần lặp và tài nguyên tính toán. Giá trị $G_{\max}$ cần được lựa chọn phù hợp với bài toán: nếu $G_{\max}$ quá nhỏ, thuật toán có thể chưa có đủ thời gian để đạt được nghiệm có chất lượng tốt; ngược lại, nếu quá lớn, thuật toán có thể tiếp tục thực hiện nhiều thế hệ dù mức độ cải thiện đã không còn đáng kể.
- **Ngưỡng độ thích nghi mục tiêu:** thuật toán có thể kết thúc khi tìm được một cá thể có độ thích nghi đạt hoặc vượt qua một ngưỡng xác định trước. Tiêu chí này phù hợp khi có thể xác định trước mức chất lượng nghiệm mong muốn hoặc có một giá trị tham chiếu để làm cơ sở đánh giá.
- **Độ trễ hội tụ:** thuật toán có thể kết thúc khi độ thích nghi của cá thể tốt nhất không cải thiện đáng kể trong K thế hệ liên tiếp. Tiêu chí này dựa trên quan sát về mức độ tiến bộ của quá trình tiến hóa. Tuy nhiên, nếu K được đặt quá nhỏ, thuật toán có thể kết thúc trong một giai đoạn mà mức cải thiện tạm thời thấp, trong khi quá trình tìm kiếm vẫn còn khả năng tiếp tục cải thiện nghiệm ở các thế hệ sau.

1.5 Lập trình Di truyền

Lập trình Di truyền (GP) được John Koza phát triển từ cuối thập niên 1980 và được hệ thống hóa trong các công trình của ông, trong đó có công trình năm 1994 [6]. GP phát triển từ các nguyên lý của Thuật toán Di truyền (GA), nhưng có một điểm khác biệt quan trọng về cách biểu diễn nghiệm. Trong GA, một cá thể thường là một chuỗi giá trị dùng để biểu diễn một nghiệm cụ thể, chẳng hạn một dãy số thực hoặc một hoán vị. Trong khi đó, một cá thể trong GP là một chương trình hoặc một biểu thức, và chương trình này có thể được thực thi để tạo ra kết quả cho bài toán. Nhờ vậy, GP có thể được sử dụng để tự động tìm ra chương trình hoặc biểu thức phù hợp với yêu cầu của bài toán mà không cần xác định trước chính xác cấu trúc của lời giải.

Mã hóa dạng cây biểu thức

Mỗi cá thể trong GP là một cây biểu thức. Mỗi nút trung gian (internal node) của cây là một toán tử được chọn từ một *tập toán tử* cho trước, chẳng hạn các phép toán số học $\{+, -, \times, \div\}$ hoặc hàm lượng giác $\{\sin, \cos\}$; mỗi nút lá là một biến đầu vào hoặc hằng số được chọn từ một *tập biến và hằng số* cho trước. Khi duyệt cây theo đúng cấu trúc phân cấp của nó, ta thu được một biểu thức toán học tường minh, có thể tính giá trị trên bất kỳ bộ dữ liệu đầu vào nào. Điều này cho phép toàn bộ quần thể các cây với hình dạng và nội dung khác nhau được đánh giá, so sánh một cách thống nhất như trong GA thông thường.

Việc thiết kế tập toán tử và tập biến, hằng số cần thỏa mãn hai tính chất cơ bản:

- **Tính khép kín** (closure): mọi toán tử phải chấp nhận bất kỳ giá trị nào do một toán tử khác hoặc một nút lá trả về, và luôn cho ra một kết quả hợp lệ. Ràng buộc này nảy sinh vì lai ghép và đột biến ghép các toán tử với nhau một cách ngẫu nhiên, không theo trật tự cố định nào. Chẳng hạn, nếu $\div$ xuất hiện tại một nút mà cây con ở vị trí mẫu số vừa tính ra 0 – một tình huống hoàn toàn có thể xảy ra – phép chia thông thường không xác định, khiến cả cây không tính được giá trị và cá thể đó trở nên vô nghĩa. Để tránh điều này, $\div$ thường được thay bằng *phép chia bảo vệ*: trả về 0 (hoặc một hằng số lớn) khi mẫu số bằng 0, thay vì để phép tính thất bại. Các hàm có miền xác định hẹp hơn tập số thực như $\log$ hay $\sqrt{\cdot}$ cũng cần phiên bản bảo vệ tương tự, chẳng hạn $\log(|x|)$ thay cho $\log(x)$, để luôn nhận được mọi giá trị đầu vào có thể xuất hiện trong cây.
- **Tính đủ** (sufficiency): hai tập hợp này phải chứa đủ các thành phần để biểu diễn được (ít nhất xấp xỉ) nghiệm của bài toán. Nếu dữ liệu thực sự mang quan hệ lượng giác mà tập toán tử chỉ gồm bốn phép tính số học, GP sẽ phải xấp xỉ hàm đích bằng đa thức với độ chính xác thấp.

Độ sâu tối đa của cây cũng là một tham số thiết kế quan trọng: giới hạn quá chặt sẽ thu hẹp không gian khám phá, trong khi không giới hạn đủ mức dễ dẫn đến các cây có kích thước rất lớn, tốn kém khi đánh giá và khó diễn giải.

Khởi tạo quần thể. Khác với GA nơi quần thể ban đầu chỉ là các chuỗi ngẫu nhiên, việc sinh cây ngẫu nhiên trong GP có ba chiến lược kinh điển, tất cả đều giới hạn độ sâu tối đa $d_{\max}$:

- **Full:** mọi nhánh của cây đều có độ sâu đúng bằng $d_{\max}$, tạo ra các cây cân đối, dày đặc;
- **Grow:** mỗi nhánh có thể kết thúc ở độ sâu bất kỳ không vượt quá $d_{\max}$, tạo ra các cây không đồng nhất về hình dạng;
- **Ramped half-and-half:** kết hợp hai phương pháp trên theo tỷ lệ 1 : 1, đồng thời phân bố độ sâu tối đa của từng cây đều trên khoảng $[2, d_{\max}]$. Đây là chiến lược được khuyến nghị và sử dụng phổ biến nhất, vì nó tạo ra quần thể ban đầu đa dạng cả về kích thước lẫn hình dạng, giảm nguy cơ quần thể bị chi phối bởi một kiểu cấu trúc duy nhất ngay từ đầu.

Hàm độ thích nghi

Một trong số những ứng dụng của GP là bài toán **hồi quy symbolic** (symbolic regression): tự động tìm một biểu thức toán học mô tả tốt mối quan hệ giữa tập dữ liệu quan sát (X, y) mà không giả định trước dạng hàm. Bài toán này khác biệt căn bản so với hồi quy tuyến tính hay hồi quy đa thức, nơi dạng hàm đã được ấn định sẵn và công việc còn lại chỉ là tối ưu các hệ số. Trong GP, độ thích nghi của một cây thường được xây dựng từ sai số dự đoán, tiêu biểu là sai số bình phương trung bình giữa giá trị cây tính được và giá trị y thực tế. Một số công trình còn dùng RMSE, MAE, hoặc hệ số xác định R^2 ; lựa chọn này ảnh hưởng trực tiếp đến độ nhạy của chọn lọc đối với các ngoại lai trong dữ liệu.

Một vấn đề đặc trưng trong GP là hiện tượng **bloat**, trong đó kích thước của các cây chương trình có xu hướng tăng qua các thế hệ mà không mang lại sự cải thiện tương xứng về độ chính xác của nghiệm. Một trong những nguyên nhân được đề xuất là sự tích lũy của các cây con trung tính, tức là những phần của chương trình có thể thay đổi nhưng không làm thay đổi đáng kể kết quả đầu ra. Ví dụ, trong một biểu thức, các thành phần như $+0$ hoặc $\times 1$ không làm thay đổi giá trị của biểu thức nhưng vẫn làm tăng kích thước cây. Do những thành phần này không làm giảm độ thích nghi, chúng có thể được duy trì và tiếp tục tích lũy qua nhiều thế hệ. Hiện tượng bloat làm tăng chi phí tính toán khi đánh giá các cá thể, khiến chương trình trở nên phức tạp và khó diễn giải hơn, đồng thời có thể làm tăng nguy cơ quá khớp. Một cách phổ biến để hạn chế bloat là đưa kích thước hoặc độ phức tạp của cây vào quá trình đánh giá,

chẳng hạn bằng cách thêm một **số hạng phạt** vào hàm thích nghi đối với những cây có kích thước lớn:

$$\text{fitness}(T) = \text{MSE}(T) + \lambda \cdot |T|, \quad (1.6)$$

trong đó $|T|$ biểu thị kích thước của cây chương trình T , thường được xác định bằng số lượng nút trong cây. Thành phần $\lambda \cdot |T|$ đóng vai trò là một khoản phạt đối với những cây có kích thước lớn, qua đó khuyến khích GP ưu tiên các chương trình có cấu trúc đơn giản và gọn nhẹ. Tham số $\lambda > 0$ kiểm soát mức độ của khoản phạt này, từ đó điều chỉnh sự đánh đổi giữa độ chính xác và độ phức tạp của chương trình. Khi λ nhỏ, quá trình tiến hóa chủ yếu tập trung vào việc giảm sai số MSE; ngược lại, khi λ lớn, các cây có kích thước lớn sẽ chịu mức phạt cao hơn và do đó ít được ưu tiên hơn. Nhờ vậy, việc bổ sung thành phần phạt kích thước cây giúp hạn chế sự gia tăng không cần thiết của cấu trúc chương trình và góp phần kiểm soát hiện tượng bloat. Các biện pháp bổ sung gồm giới hạn độ sâu/số nút tuyệt đối, loại bỏ cá thể vi phạm ngay sau khi sinh, và các toán tử đột biến có chủ đích rút gọn cây.

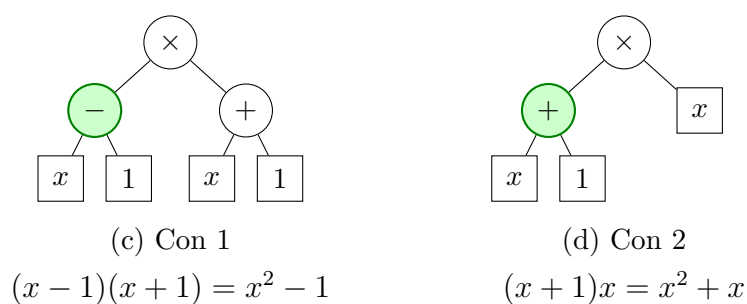
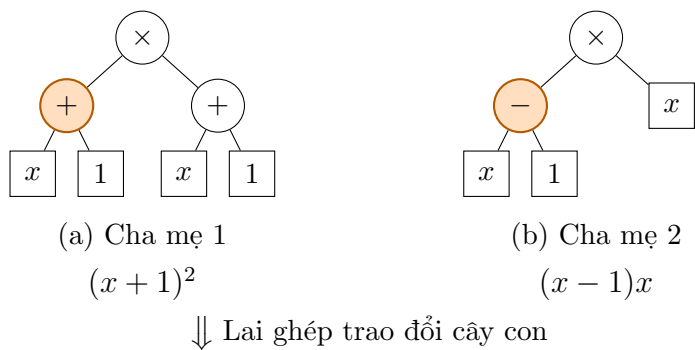
Toán tử tiến hóa

Các toán tử lai ghép và đột biến trong GP thao tác trực tiếp trên cấu trúc cây thay vì trên chuỗi có độ dài cố định:

- **Lai ghép trao đổi cây con:** chọn ngẫu nhiên một nút trên mỗi cây cha mẹ, sau đó hoán đổi hai cây con bắt rễ tại các nút đó cho nhau để tạo ra hai cây con mới. Các biến thể gồm lai ghép bất đối xứng (cây con thứ nhất lấy từ nút trung gian, cây con thứ hai từ nút lá) và lai ghép tự thân (cắt và dán trên cùng một cây), nhằm cân bằng giữa khám phá và khai thác;
- **Đột biến thay thế cây con:** chọn ngẫu nhiên một nút trên cây, rồi thay toàn bộ cây con bắt rễ tại nút đó bằng một cây con mới được sinh ngẫu nhiên;
- **Đột biến điểm** (point mutation): thay một nút hàm bằng một hàm khác cùng số ngôi, hoặc thay một nút lá bằng biến/hằng số khác, giữ nguyên cấu trúc cây;
- **Đột biến rút gọn** (hoist mutation): thay toàn bộ cây bằng một cây con ngẫu nhiên bên trong nó, qua đó thu nhỏ cây một cách có kiểm soát – biến thể này thường được dùng kết hợp với số hạng phạt (1.6) để chống bloat.

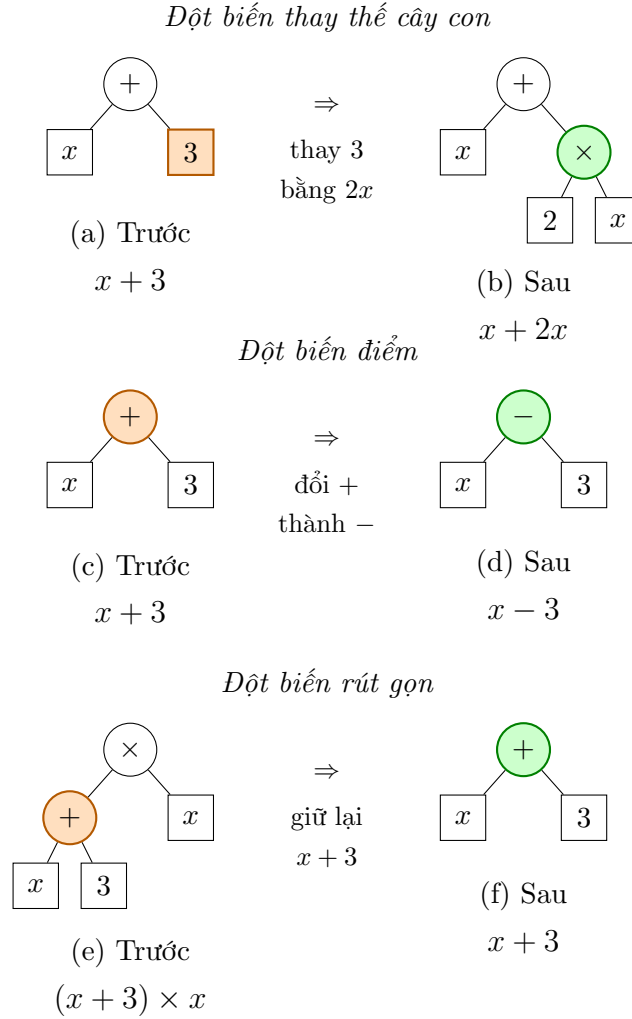
Các thành phần còn lại của chu trình tiến hóa, bao gồm khởi tạo quần thể, chọn lọc, elitism và điều kiện dừng, về bản chất tương tự như GA đã trình bày trong các mục trước; khác biệt duy nhất nằm ở đối tượng thao tác là **cây** thay vì **vector** số hoặc hoán vị. Điều kiện dừng thường là số thế hệ tối đa kết hợp với ngưỡng sai số chấp nhận được.

Hình 1.7 minh họa phép lai ghép trao đổi cây con giữa hai cây cha mẹ $(x+1)^2$ và $(x-1)x$ với tập toán tử $\{+, -, \times\}$.



Hình 1.7: Lai ghép trao đổi cây con giữa hai cây cha mẹ (a)–(b), tạo ra hai cây con (c)–(d).

Hình 1.8 minh họa ba toán tử đột biến trên cây đơn giản $x+3$ (và, với đột biến rút gọn, trên cây lớn hơn $(x+3) \times x$).



Hình 1.8: Ba toán tử đột biến minh họa trên cây $x + 3$ (đột biến rút gọn minh họa trên cây lớn hơn $(x + 3) \times x$): (a)–(b) đột biến thay thế cây con, (c)–(d) đột biến điểm, (e)–(f) đột biến rút gọn.

Tóm lại, chương này đã trình bày đầy đủ các nền tảng lý thuyết của GA và GP. Trên cơ sở đó, Chương 2 sẽ trình bày việc áp dụng GA cho ba nhóm bài toán đầu tiên – tối ưu không ràng buộc, tối ưu có ràng buộc, và bài toán người du lịch – cùng với việc áp dụng GP cho bài toán hồi quy symbolic trên dữ liệu quan trắc thời tiết thực tế, dự đoán nhiệt độ điểm sương từ nhiệt độ không khí và độ ẩm tương đối. Kết quả thực nghiệm của ba nhóm bài toán GA sẽ được đối chiếu với một mốc so sánh tương ứng: giá trị cực tiểu toàn cục đã biết của các hàm benchmark cho bài toán không ràng buộc, nghiệm tối ưu công bố của bộ benchmark CEC 2006 cho bài toán có ràng buộc, và nghiệm tối ưu công bố của TSPLIB95 cho bài toán người du lịch. Riêng kết quả của GP cho bài toán hồi quy symbolic sẽ được đối chiếu với công thức Magnus – quan hệ vật lý đúng đã biết giữa ba đại lượng – nhằm đánh giá mức độ GP tái khám phá được cấu trúc thật của quan hệ này.

Chương 2

Thực nghiệm

Chương này trình bày việc áp dụng các nguyên lý đã trình bày ở Chương 1 vào bốn nhóm bài toán cụ thể. Mỗi nhóm sử dụng một cách mã hóa khác nhau: tối ưu không ràng buộc dùng mã hóa số thực (Mục 2.1), tối ưu có ràng buộc cũng dùng mã hóa số thực nhưng kết hợp thêm cơ chế xử lý ràng buộc (Mục 2.2), bài toán người du lịch dùng mã hóa hoán vị (Mục 2.3), và hồi quy symbolic dùng mã hóa dạng cây biểu thức theo nguyên lý Lập trình Di truyền (Mục 2.4). Với mỗi bài toán, phát biểu toán học đầy đủ được trình bày trước, sau đó là phương pháp so sánh được sử dụng và bảng kết quả thực nghiệm.

2.1 Tối ưu không ràng buộc

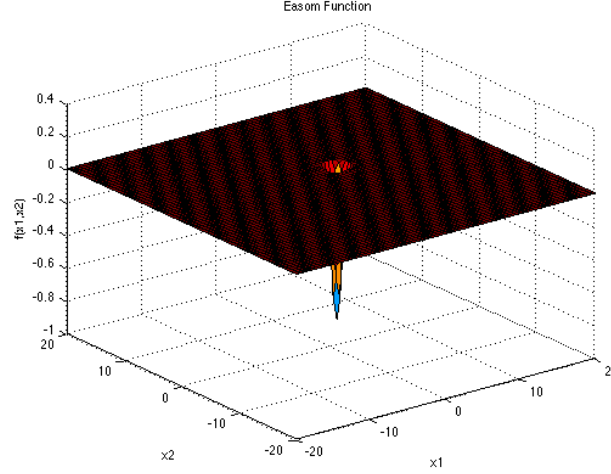
Năm hàm benchmark kinh điển của tối ưu không ràng buộc được sử dụng để đánh giá khả năng tìm cực trị toàn cục của GA. Cả năm hàm đều có hai biến x, y và một cực tiểu toàn cục đã biết trước, cho phép so sánh trực tiếp giá trị GA tìm được với giá trị đúng. Với mỗi hàm, đồ thị ba chiều và đường hội tụ của GA được đặt trên dưới nhau trong cùng một hình. Miền tìm kiếm của từng hàm được lấy theo bộ giá trị chuẩn công bố tại <https://www.sfu.ca/~ssurjano/optimization.html> [7], nơi người đọc có thể tham khảo thêm công thức và đặc trưng chi tiết của từng hàm benchmark.

Hàm Easom có công thức

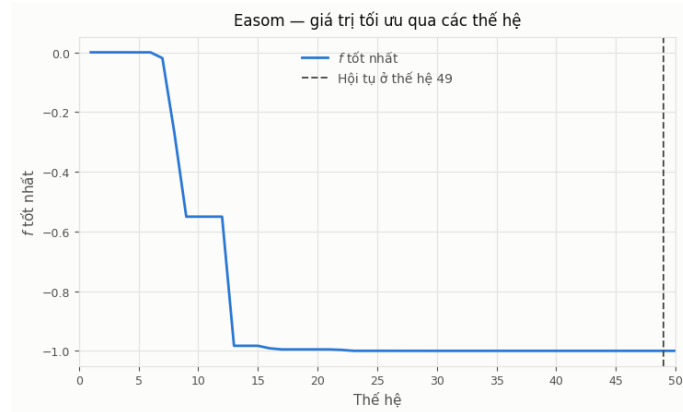
$$f(x, y) = -\cos(x) \cos(y) \exp(-(x - \pi)^2 - (y - \pi)^2), \quad (2.1)$$

với miền tìm kiếm $x, y \in [-100, 100]$ và cực tiểu toàn cục $f(\pi, \pi) = -1$. Khó khăn của hàm này nằm ở chỗ thừa số $\exp(-(x - \pi)^2 - (y - \pi)^2)$ giảm rất nhanh về 0 khi rời khỏi điểm (π, π) : chỉ cần cách tâm khoảng ba đơn vị, giá trị hàm đã nhỏ hơn 10^{-4} và coi như bằng 0. Vùng thực sự mang thông tin dẫn đường vì thế chiếm chưa tới 0.1% diện tích miền tìm kiếm $[-100, 100]^2$, toàn bộ phần còn lại gần như là một mặt phẳng

tuyệt đối. Ở vùng phẳng đó, đạo hàm theo mọi hướng đều xấp xỉ 0, nên các phương pháp dựa trên gradient không có cơ sở nào để xác định nên di chuyển về đâu — đây là tình huống bất lợi nhất đối với nhóm phương pháp này. Hàm Easom vì vậy kiểm tra đúng một khả năng: rải điểm đủ rộng để tìm ra vùng chứa nghiệm, chứ không phải khả năng tinh chỉnh sau khi đã vào gần nghiệm.



(a) Hàm Easom trong không gian ba chiều



(b) Đường hội tụ của GA áp dụng trên hàm Easom

Hình 2.1: Hàm Easom: đồ thị trong không gian ba chiều và diễn biến giá trị tốt nhất qua các thế hệ. Đường nét đứt đánh dấu thế hệ mà GA chốt được nghiệm tốt nhất.

Hàm Rastrigin có công thức

$$f(x, y) = 20 + x^2 + y^2 - 10 \cos(2\pi x) - 10 \cos(2\pi y), \quad (2.2)$$

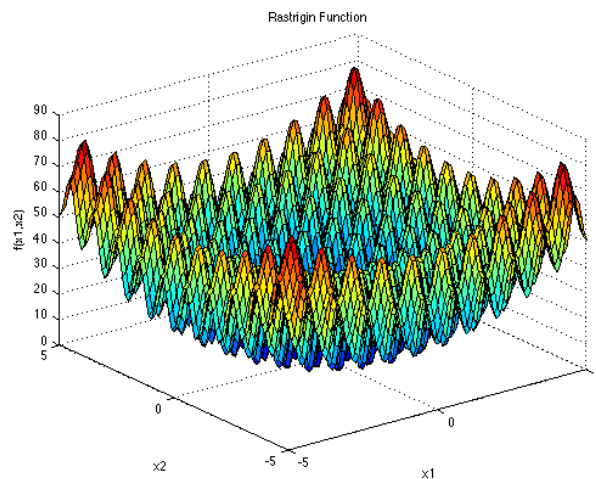
với miền tìm kiếm $x, y \in [-5.12, 5.12]$ và cực tiểu toàn cục $f(0, 0) = 0$.

Bề mặt của hàm không trơn theo nghĩa trực quan mà gồm rất nhiều gợn sóng liên tiếp, tạo thành các cực tiểu và cực đại cục bộ xen kẽ nhau. Do hai số hạng cosin có chu kỳ bằng 1, các cực tiểu cục bộ này xuất hiện gần các điểm có tọa độ nguyên; trong

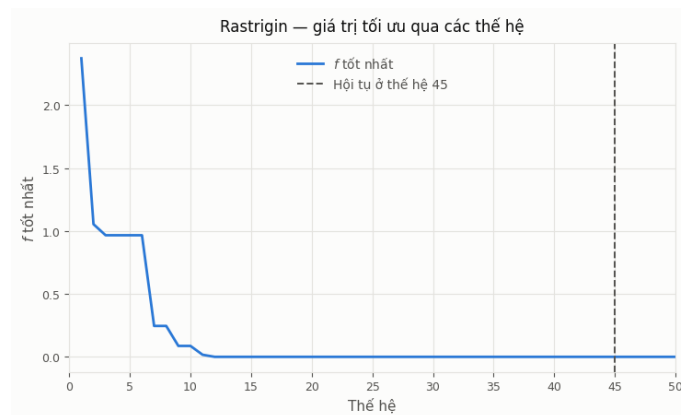
miền $[-5.12, 5.12]^2$ có khoảng 121 cực tiểu cục bộ như vậy, bao quanh dày đặc cực tiểu toàn cục tại gốc tọa độ.

Cấu trúc này khiến Rastrigin trở thành bài toán *đa cực trị* điển hình, khó đối với các phương pháp chỉ tìm kiếm trong phạm vi lân cận: xuất phát từ một điểm ngẫu nhiên, phương pháp cực bộ gần như chắc chắn hội tụ về cực tiểu cục bộ gần nhất rồi dừng lại tại đó, dù vẫn còn một nghiệm tốt hơn ở nơi khác trong không gian tìm kiếm. Nói cách khác, thuật toán dễ bị mắc kẹt tại một cực tiểu cục bộ thay vì tìm được cực tiểu toàn cục.

Đặc điểm này cũng chính là lý do Rastrigin phù hợp để minh họa ưu điểm của GA. Thay vì chỉ cải thiện một nghiệm duy nhất, GA duy trì đồng thời nhiều cá thể trong quần thể và liên tục sinh ra nghiệm mới qua lai ghép và đột biến; đột biến với biên độ phù hợp có thể đưa cá thể thoát khỏi vùng đang bị mắc kẹt để khám phá các vùng khác của không gian tìm kiếm. Nhờ đó, GA có khả năng vượt qua các cực tiểu cục bộ và tăng cơ hội tìm được cực tiểu toàn cục tại gốc tọa độ.



(a) Hàm Rastrigin trong không gian ba chiều



(b) Đường hội tụ của GA áp dụng trên hàm Rastrigin

Hình 2.2: Hàm Rastrigin: mạng lưới cực tiểu cục bộ dày đặc quanh cực tiểu toàn cục, và diễn biến giá trị tốt nhất qua các thế hệ.

Hàm Rosenbrock có công thức

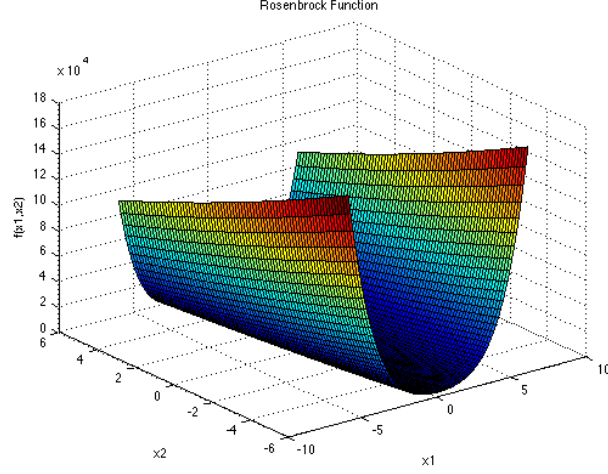
$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2, \quad (2.3)$$

với miền tìm kiếm $x, y \in [-5, 10]$ và cực tiểu toàn cục $f(1, 1) = 0$.

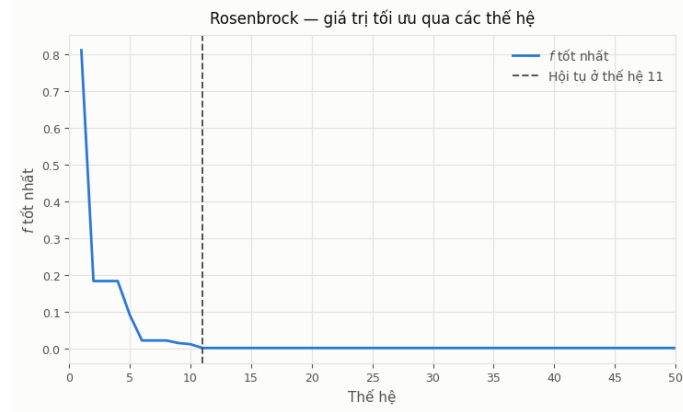
Khác với bốn hàm còn lại, Rosenbrock không có nhiều cực tiểu cục bộ trong miền xét; khó khăn ở đây không nằm ở việc thuật toán bị mắc kẹt tại một cực tiểu cục bộ, mà nằm ở hình dạng của vùng dẫn đến nghiệm tối ưu. Số hạng $100(y - x^2)^2$ phạt rất nặng mọi điểm (x, y) lệch khỏi đường cong $y = x^2$, khiến vùng có giá trị hàm thấp co lại thành một thung lũng dài, hẹp và cong, với nghiệm tối ưu $(1, 1)$ nằm ở đáy.

Dọc theo hai bên thung lũng, giá trị hàm tăng rất nhanh khi lệch khỏi đường cong $y = x^2$, trong khi dọc theo đáy thung lũng giá trị hàm lại thay đổi rất chậm. Vì vậy, thuật toán phải thay đổi x và y một cách *phối hợp* theo đúng đường cong này để tiến dần về $(1, 1)$, thay vì chỉ cải thiện từng biến một cách độc lập.

Do đó, Rosenbrock kiểm tra một khả năng khác của GA so với Rastrigin: nếu Rastrigin kiểm tra khả năng thoát khỏi một mạng lưới dày đặc các cực tiểu cục bộ, thì Rosenbrock kiểm tra khả năng *dò theo* một không gian hẹp, cong và khó tiếp cận sau khi đã tìm ra đúng vùng chứa nghiệm. Đây là ví dụ điển hình cho trường hợp bài toán không có nhiều bẫy cục bộ nhưng vẫn khó tối ưu do cấu trúc của không gian tìm kiếm.



(a) Hàm Rosenbrock trong không gian ba chiều



(b) Đường hội tụ của GA áp dụng trên hàm Rosenbrock

Hình 2.3: Hàm Rosenbrock: thung lũng hẹp và cong, và diễn biến giá trị tốt nhất qua các thế hệ — GA chốt kỷ lục ngay ở thế hệ 11 rồi không cải thiện thêm trong 39 thế hệ còn lại.

Hàm Ackley có công thức

$$f(x, y) = -20 \exp\left(-0.2\sqrt{0.5(x^2 + y^2)}\right) - \exp(0.5 \cos(2\pi x) + 0.5 \cos(2\pi y)) + 20 + e, \quad (2.4)$$

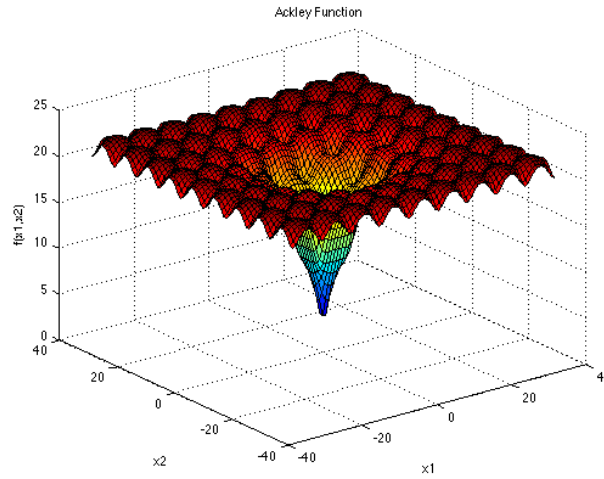
với miền tìm kiếm $x, y \in [-32.768, 32.768]$ và cực tiểu toàn cục $f(0, 0) = 0$.

Bề mặt của hàm có thể hình dung như một phễu lớn phủ nhiều gợn sóng nhỏ. Số hạng mũ thứ nhất tạo ra xu thế tổng thể hướng về gốc tọa độ: càng xa $(0, 0)$, giá trị hàm nhìn chung càng lớn — đây là thông tin có ích, vì nó chỉ đúng hướng cần tiến về nghiệm. Số hạng cosin phủ thêm lên bề mặt đó vô số gợn sóng và cực tiểu cục bộ nhỏ, khiến quá trình tìm kiếm có thể bị nhiễu hoặc dừng lại tại những vị trí chưa tối ưu.

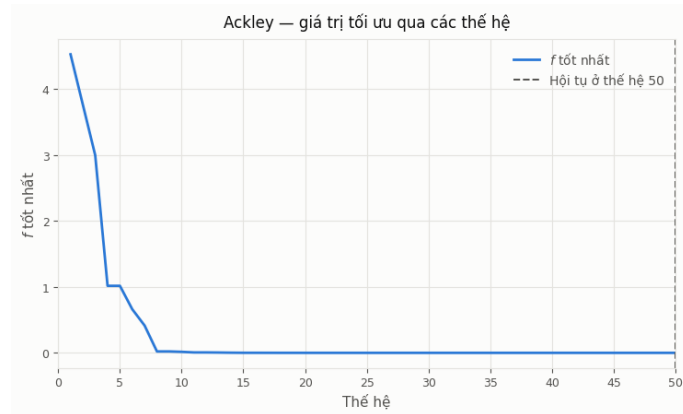
Do đó, khó khăn của Ackley nằm ở việc phân biệt xu thế toàn cục giữa nhiều biến

động cục bộ: thuật toán vừa phải tận dụng xu thế chung hướng về gốc tọa độ, vừa phải duy trì khả năng khám phá để không bị mắc kẹt bởi các gợn sóng nhỏ. Với GA, lai ghép và đột biến liên tục tạo ra cá thể mới ở những vị trí khác nhau, qua đó tăng khả năng vượt qua các cực tiểu cục bộ và tiếp tục tiến về vùng tối ưu.

Như vậy, nếu Rastrigin kiểm tra khả năng thoát khỏi một mạng lưới dày đặc các cực tiểu cục bộ, còn Rosenbrock kiểm tra khả năng dò theo một thung lũng hẹp và cong, thì Ackley kiểm tra khả năng nhận diện xu thế tối ưu ở quy mô lớn trong khi không bị các biến động nhỏ ở quy mô cục bộ chi phối.



(a) Hàm Ackley trong không gian ba chiều



(b) Đường hội tụ của GA áp dụng trên hàm Ackley

Hình 2.4: Hàm Ackley: phễu lớn ở thang toàn cục phủ gợn sóng cực tiểu cục bộ ở thang nhỏ, và diễn biến giá trị tốt nhất qua các thế hệ.

Hàm Schwefel có công thức

$$f(x, y) = 837.9658 - x \sin \sqrt{|x|} - y \sin \sqrt{|y|}, \quad (2.5)$$

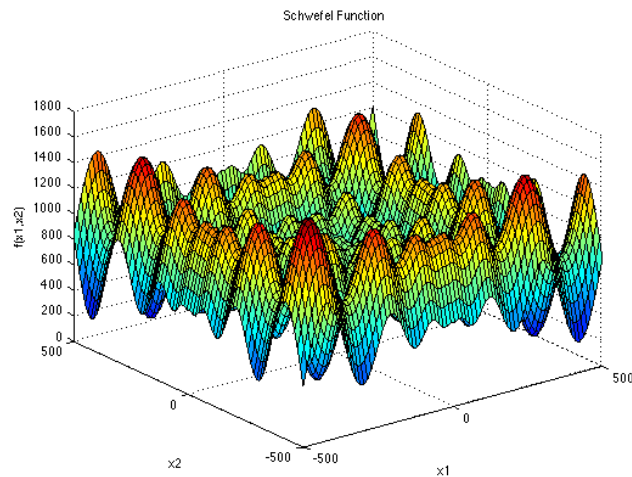
với miền tìm kiếm $x, y \in [-500, 500]$ và cực tiểu toàn cục $f(420.9687, 420.9687) \approx 0$.

Đặc điểm của Schwefel là vùng có vẻ tốt nhất tại một vị trí có thể không dẫn đến

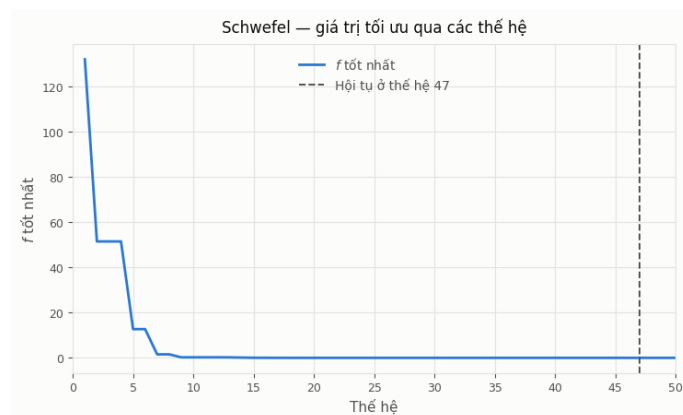
nghiệm tối ưu thực sự: các cực tiểu cục bộ của nó phân bố không đều trong toàn miền tìm kiếm, và cực tiểu cục bộ tốt thứ nhì lại nằm rất xa cực tiểu toàn cục. Do đó, nếu thuật toán chỉ tập trung khai thác một vùng đang cho kết quả tốt, nó có thể nhanh chóng đạt được một nghiệm khá tốt nhưng bỏ qua nghiệm tốt nhất nằm ở một khu vực hoàn toàn khác.

Thêm vào đó, cực tiểu toàn cục $(420.9687, 420.9687)$ nằm khá gần biên của miền tìm kiếm, nên thuật toán không nên mặc định rằng nghiệm tốt nằm gần tâm không gian tìm kiếm; nếu cơ chế xử lý biên vô tình co cá thể về phía tâm miền, khả năng tìm thấy nghiệm tối ưu sẽ giảm.

Vì vậy, Schwefel chủ yếu kiểm tra khả năng duy trì sự đa dạng và thăm dò trên phạm vi rộng của không gian tìm kiếm, thay vì hội tụ sớm vào một vùng có vẻ hứa hẹn. Đây là một điểm mạnh của GA, vì quần thể gồm nhiều cá thể cho phép thuật toán tiếp tục khám phá nhiều khu vực khác nhau thay vì chỉ đi theo một hướng duy nhất.



(a) Hàm Schwefel trong không gian ba chiều



(b) Đường hội tụ của GA áp dụng trên hàm Schwefel

Hình 2.5: Hàm Schwefel: cực tiểu toàn cục nằm gần biên miền tìm kiếm, xa vùng trông có vẻ tốt quanh gốc tọa độ, và diễn biến giá trị tốt nhất qua các thế hệ.

Đối với mỗi hàm benchmark, GA được thực nghiệm trên cùng một thiết lập với quần thể 500 cá thể và tối đa 50 thế hệ. Các cá thể được giữ trong miền tìm kiếm của từng hàm và thuật toán luôn chạy đủ 50 thế hệ. Cột *số thế hệ thỏa mãn* biểu thị thế hệ sớm nhất đạt được giá trị tốt nhất cuối cùng f^* , không phải thời điểm thuật toán dừng.

Bảng 2.1: Kết quả GA trên năm hàm benchmark không ràng buộc

Hàm	Thời gian hội tụ (s)	Thế hệ	$f_{\text{thực}}$	f^*	x^*	y^*
Easom	0.71	48	-1	-1.0000000000	3.141593	3.141593
Rastrigin	0.77	45	0	0.0000000000	8.00×10^{-10}	-2.11×10^{-9}
Rosenbrock	0.70	11	0	0.0009051359	1.008839	1.014881
Ackley	0.75	50	0	3.344×10^{-10}	-7.55×10^{-11}	-9.10×10^{-11}
Schwefel	0.73	47	≈ 0	2.5455×10^{-5}	420.968746	420.968746

Nhận xét và đánh giá kết quả

Kết quả cho thấy GA tìm được nghiệm toàn cục hoặc nghiệm rất gần tối ưu trên cả năm hàm. Easom và Rastrigin đạt đúng giá trị tối ưu; Ackley và Schwefel cũng đạt sai số rất nhỏ. Đáng chú ý, GA vẫn tìm được nghiệm tốt trên Easom và Schwefel, cho thấy khả năng khám phá rộng của tìm kiếm dựa trên quần thể.

Rosenbrock là trường hợp khó nhất khi chỉ đạt $f^* = 9.05 \times 10^{-4}$. GA nhanh chóng tiến vào vùng gần nghiệm tối ưu nhưng sau đó hầu như không cải thiện thêm. Nguyên nhân là thung lũng của Rosenbrock hẹp và cong, đòi hỏi quá trình tinh chỉnh chính xác dọc theo đáy thung lũng, trong khi các phép biến đổi của GA không thực sự hiệu quả ở giai đoạn này.

Nhìn chung, kết quả cho thấy GA có ưu thế rõ rệt trong khám phá và định vị vùng nghiệm, nhưng khả năng tinh chỉnh nghiệm trong những vùng hẹp và có cấu trúc đặc biệt còn hạn chế. Đây cũng là cơ sở để kết hợp GA với tìm kiếm cục bộ trong các bài toán cần độ chính xác cao ở giai đoạn cuối.

2.2 Tối ưu có ràng buộc

Với bài toán có ràng buộc, không phải mọi điểm trong miền tìm kiếm đều hợp lệ; do đó, thuật toán phải xét tính khả thi trước khi tối ưu hàm mục tiêu, vì nghiệm vi phạm ràng buộc dù có giá trị mục tiêu tốt vẫn không có ý nghĩa.

Thực nghiệm sử dụng bộ benchmark CEC 2006 của Liang và cộng sự (https://cw.fel.cvut.cz/b241/_media/courses/a0m33eoa/cviceni/2006_problem_definitions_and_evaluation_criteria_for_the_cec_2006_special_session_on_constraint_real-parameter_optimization.pdf [8]), gồm 24 bài toán tối ưu có ràng buộc kèm nghiệm tối ưu. Năm bài toán g01, g06, g08, g11 và g15 được chọn nhằm bao

quát nhiều dạng ràng buộc, mức độ chặt hẹp của miền khả thi và vị trí của nghiệm tối ưu so với biên.

Theo bộ benchmark CEC 2006, ràng buộc bất đẳng thức yêu cầu $g_i(x) \leq 0$, trong khi ràng buộc đẳng thức sử dụng dung sai $|h_j(x)| - \varepsilon \leq 0$ với $\varepsilon = 10^{-4}$. Với ràng buộc đẳng thức, việc yêu cầu $h_j(x) = 0$ một cách tuyệt đối là không thực tế đối với thuật toán số, vì chỉ cần sai lệch rất nhỏ cũng khiến nghiệm bị xem là không hợp lệ. Chẳng hạn, nghiệm công bố của g11 và g15 cũng nằm ngay tại mức sai số này.

GA mã hóa số thực được kết hợp quy tắc khả thi của Deb và ngưỡng ε giảm dần (Mục 1.3), sử dụng 1000 cá thể trong 500 thế hệ. Sau GA, một bước tìm kiếm cục bộ theo từng trục tọa độ được thực hiện từ nghiệm tốt nhất: chỉ chấp nhận điểm khả thi có giá trị mục tiêu tốt hơn; khi không còn cải thiện, bước dịch chuyển được giảm một nửa và tiếp tục tìm kiếm. Mô hình kết hợp này cũng được áp dụng cho bài toán người du lịch ở Mục 2.3. Trong đó, GA đảm nhiệm khám phá toàn cục, còn tìm kiếm cục bộ đảm nhiệm tinh chỉnh nghiệm, đặc biệt khi nghiệm nằm sát biên miền khả thi. Kết quả của cả hai giai đoạn được báo cáo riêng để đánh giá đóng góp của từng bước. Kết quả chi tiết trên từng bài toán được trình bày lần lượt dưới đây.

Bài toán g01 được phát biểu như sau:

$$\begin{aligned}
& \min_{x \in \mathbb{R}^{13}} \quad f(x) = 5 \sum_{i=1}^4 x_i - 5 \sum_{i=1}^4 x_i^2 - \sum_{i=5}^{13} x_i \\
& \text{subject to} \quad 2x_1 + 2x_2 + x_{10} + x_{11} - 10 \leq 0 \\
& \quad \quad \quad 2x_1 + 2x_3 + x_{10} + x_{12} - 10 \leq 0 \\
& \quad \quad \quad 2x_2 + 2x_3 + x_{11} + x_{12} - 10 \leq 0 \\
& \quad \quad \quad -8x_1 + x_{10} \leq 0 \\
& \quad \quad \quad -8x_2 + x_{11} \leq 0 \\
& \quad \quad \quad -8x_3 + x_{12} \leq 0 \\
& \quad \quad \quad -2x_4 - x_5 + x_{10} \leq 0 \\
& \quad \quad \quad -2x_6 - x_7 + x_{11} \leq 0 \\
& \quad \quad \quad -2x_8 - x_9 + x_{12} \leq 0
\end{aligned} \tag{2.6}$$

với miền biến thiên $0 \leq x_i \leq 1$ cho $i = 1, \dots, 9$, $0 \leq x_i \leq 100$ cho $i = 10, 11, 12$ và $0 \leq x_{13} \leq 1$. Nghiệm tối ưu công bố là

$$x^* = (1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 3, 3, 1), \quad f(x^*) = -15,$$

tại đó sáu trong chín ràng buộc đạt đẳng thức.

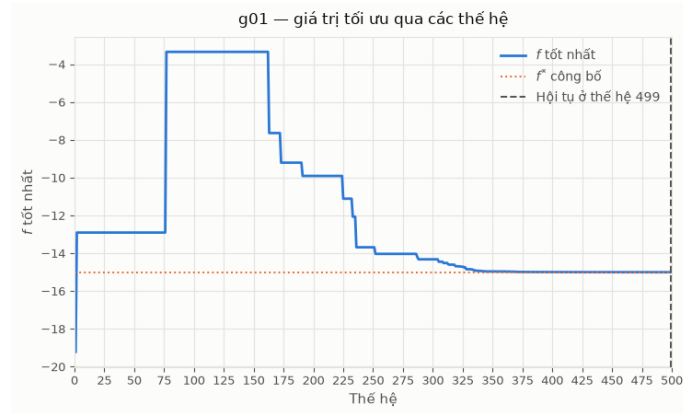
g01 là bài toán có số biến lớn nhất trong năm bài được xét, với mười ba biến và chín ràng buộc bất đẳng thức tuyến tính. Do đó, miền khả thi có cấu trúc đa diện,

không xuất hiện các ràng buộc phi tuyến gây biến dạng miền nghiệm.

Bảng 2.2: Kết quả trên bài toán g01

f^* công bố	f^* (GA)	Sai lệch (GA)	f^* (GA + tinh chỉnh)	Sai lệch (GA + tinh chỉnh)	Số thể hệ thỏa mãn
-15.000000	-14.999204	7.96×10^{-4}	-15.000000	1.77×10^{-12}	499

GA đơn thuần đạt sai lệch tương đối 5.3×10^{-5} so với nghiệm tối ưu công bố. Sau bước tinh chỉnh cục bộ, sai lệch giảm xuống 1.77×10^{-12} , cho thấy bước tinh chỉnh cải thiện đáng kể độ chính xác của nghiệm.



Hình 2.6: Đường hội tụ của GA trên bài toán g01

Hình 2.6 cho thấy quá trình hội tụ trên g01 có thể chia thành hai giai đoạn. Trước thế hệ 163, giá trị tốt nhất biến động do chưa xuất hiện cá thể thỏa mãn hoàn toàn các ràng buộc, nên giá trị hàm mục tiêu chưa phản ánh trực tiếp chất lượng tối ưu. Từ thế hệ 163, giá trị tốt nhất giảm dần và tiệm cận $f^* = -15$, sau đó chỉ cải thiện rất nhỏ cho đến thế hệ 499. Điều này cho thấy GA cần số thế hệ tương đối lớn để đồng thời thỏa mãn chín ràng buộc và đạt độ chính xác cao trong không gian mười ba chiều.

Bài toán g06 được phát biểu như sau:

$$\begin{aligned}
 \min_{x \in \mathbb{R}^2} \quad & f(x) = (x_1 - 10)^3 + (x_2 - 20)^3 \\
 \text{subject to} \quad & -(x_1 - 5)^2 - (x_2 - 5)^2 + 100 \leq 0 \\
 & (x_1 - 6)^2 + (x_2 - 5)^2 - 82.81 \leq 0
 \end{aligned} \tag{2.7}$$

với $13 \leq x_1 \leq 100$ và $0 \leq x_2 \leq 100$. Nghiệm tối ưu công bố là

$$x^* = (14.095000; 0.842961), \quad f(x^*) = -6961.813876,$$

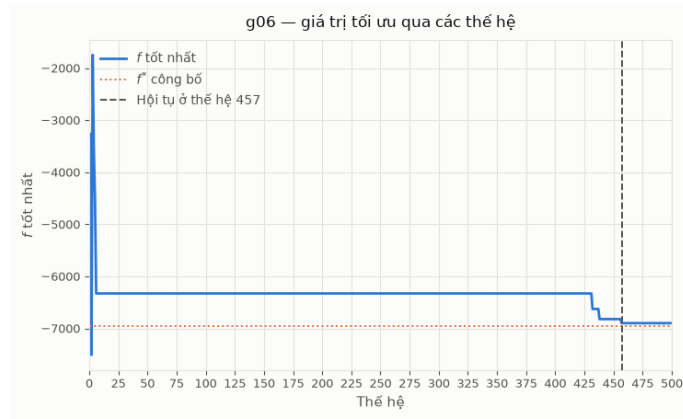
tại đó cả hai ràng buộc đều đạt đẳng thức.

g06 là bài toán chỉ có hai biến nhưng hàm mục tiêu bậc ba và cả hai ràng buộc đều phi tuyến. Đặc biệt, nghiệm tối ưu nằm tại điểm mà cả hai ràng buộc đều đạt đẳng thức, do đó việc vừa duy trì tính khả thi vừa tìm kiếm nghiệm tối ưu là tương đối khó.

Bảng 2.3: Kết quả trên bài toán g06

f^* công bố	f^* (GA)	Sai lệch (GA)	f^* (GA + tinh chỉnh)	Sai lệch (GA + tinh chỉnh)	Số thế hệ thỏa mãn
-6961.813876	-6898.462524	63.35	-6961.813876	2.64×10^{-8}	457

Theo Bảng 2.3, GA đơn thuần đạt $f^* = -6898.462524$, sai lệch 63.35 so với nghiệm tối ưu công bố -6961.813876 . Sau bước tinh chỉnh cục bộ, giá trị đạt $f^* = -6961.813876$, với sai lệch chỉ 2.64×10^{-8} . Kết quả cho thấy bước tinh chỉnh cải thiện đáng kể độ chính xác của nghiệm.



Hình 2.7: Đường hội tụ của GA trên bài toán g06

Hình 2.7 cho thấy giá trị tốt nhất dao động mạnh trong hai thế hệ đầu. Nguyên nhân là khi chưa có cá thể khả thi, quy tắc lựa chọn ưu tiên **cá thể có tổng mức vi phạm ràng buộc nhỏ hơn**; vì vậy, cá thể tốt nhất có thể thay đổi giữa các thế hệ và kéo theo giá trị hàm mục tiêu dao động từ khoảng -7507 đến -1745 . Từ thế hệ 3, thuật toán tìm được cá thể khả thi và gần như hội tụ tại mức -6328.01 trong hơn bốn trăm thế hệ. Chỉ từ khoảng thế hệ 451, giá trị tốt nhất mới tiếp tục được cải thiện lên -6898.46 và duy trì đến cuối quá trình. Khoảng cách còn lại đến nghiệm tối ưu công bố -6961.81 được thu hẹp đáng kể sau bước tinh chỉnh cục bộ.

Bài toán g08 được phát biểu như sau:

$$\begin{aligned}
 \min_{x \in \mathbb{R}^2} \quad & f(x) = -\frac{\sin^3(2\pi x_1) \sin(2\pi x_2)}{x_1^3 (x_1 + x_2)} \\
 \text{subject to} \quad & x_1^2 - x_2 + 1 \leq 0 \\
 & 1 - x_1 + (x_2 - 4)^2 \leq 0
 \end{aligned} \tag{2.8}$$

với $0 \leq x_1 \leq 10$ và $0 \leq x_2 \leq 10$. Nghiệm tối ưu công bố là

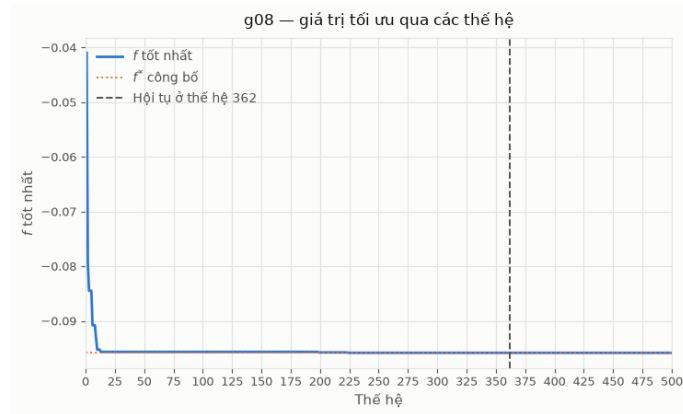
$$x^* = (1.227971; 4.245373), \quad f(x^*) = -0.095825.$$

g08 là bài toán hai biến với hàm mục tiêu phi tuyến, chứa cả hàm lượng giác và phân thức, cùng hai ràng buộc bất đẳng thức phi tuyến.

Bảng 2.4: Kết quả trên bài toán g08

f^* công bố	f^* (GA)	Sai lệch (GA)	f^* (GA + tinh chỉnh)	Sai lệch (GA + tinh chỉnh)	Số thế hệ thỏa mãn
-0.095825	-0.095825	2.78×10^{-17}	-0.095825	2.78×10^{-17}	362

Theo Bảng 2.4, GA đơn thuần đã đạt đúng giá trị tối ưu công bố $f^* = -0.095825$, với sai lệch chỉ 2.78×10^{-17} . Bước tinh chỉnh cục bộ không làm thay đổi kết quả, cho thấy GA đã xác định được nghiệm với độ chính xác rất cao. Thuật toán đạt giá trị tối ưu tại thế hệ 362.



Hình 2.8: Đường hội tụ của GA trên bài toán g08

Hình 2.8 cho thấy quá trình hội tụ diễn ra tương đối ổn định. Giá trị tốt nhất giảm nhanh trong khoảng hai mươi lăm thế hệ đầu, từ khoảng -0.041 xuống gần -0.09565 . Sau đó, giá trị chỉ tiếp tục cải thiện qua các bước rất nhỏ và gần như giữ nguyên quanh mức này trong phần lớn số thế hệ, trước khi đạt -0.095825 tại thế hệ 362. Sự ổn định ở giai đoạn cuối cho thấy quần thể đã tập trung quanh vùng nghiệm tốt và không còn tạo ra các cá thể có giá trị cải thiện đáng kể.

Bài toán g11 được phát biểu như sau:

$$\begin{aligned} \min_{x \in \mathbb{R}^2} \quad & f(x) = x_1^2 + (x_2 - 1)^2 \\ \text{subject to} \quad & x_2 - x_1^2 = 0 \end{aligned} \quad (2.9)$$

với $-1 \leq x_1 \leq 1$ và $-1 \leq x_2 \leq 1$. Nghiệm tối ưu công bố là

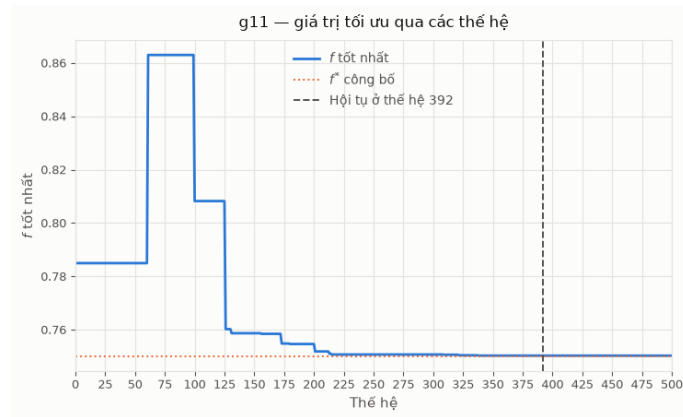
$$x^* = (-0.707036; 0.500000), \quad f(x^*) = 0.7499.$$

Miền khả thi chính xác của g11 là đường parabol $x_2 = x_1^2$; các điểm sinh ra ngẫu nhiên khi khởi tạo quần thể, hay qua lai ghép và đột biến, hầu như không bao giờ rơi đúng lên đường này. Khi áp dụng dung sai $\varepsilon = 10^{-4}$, các điểm lệch khỏi đường này một khoảng nhỏ vẫn được xem là khả thi. Nếu yêu cầu ràng buộc thỏa mãn tuyệt đối, giá trị nhỏ nhất của hàm mục tiêu là 0.75. Giá trị công bố 0.7499 thấp hơn vì nghiệm của CEC 2006 được phép lệch khỏi đường parabol trong phạm vi dung sai. GA đạt 0.750268, rất gần mức 0.75, nhưng chưa tìm được điểm tận dụng dung sai để giảm giá trị hàm mục tiêu xuống mức 0.7499. Bước tinh chỉnh cũng chỉ cải thiện rất nhỏ do miền chấp nhận quanh đường parabol rất hẹp.

Bảng 2.5: Kết quả trên bài toán g11

f^* công bố	f^* (GA)	Sai lệch (GA)	f^* (GA + tinh chỉnh)	Sai lệch (GA + tinh chỉnh)	Số thế hệ thỏa mãn
0.749900	0.750268	3.68×10^{-4}	0.750259	3.59×10^{-4}	392

Theo Bảng 2.5, GA đạt $f^* = 0.750268$, với sai lệch 3.68×10^{-4} so với nghiệm tối ưu công bố 0.749900. Sau tinh chỉnh cục bộ, sai lệch chỉ giảm nhẹ xuống 3.59×10^{-4} , cho thấy bước tinh chỉnh có tác động hạn chế đối với bài toán này. Nghiệm đạt tiêu chí thỏa mãn tại thế hệ 392.



Hình 2.9: Đường hội tụ của GA trên bài toán g11

Hình 2.9 cho thấy giá trị tốt nhất dao động trong giai đoạn đầu do quần thể chưa tìm được cá thể thỏa mãn ràng buộc $x_2 - x_1^2 = 0$ trong giới hạn dung sai, nên việc lựa chọn vẫn chịu chi phối bởi mức độ vi phạm. Điều này dẫn đến các biến động, như mức tăng từ 0.785 lên 0.863 tại thế hệ 61. Từ khoảng thế hệ 100, giá trị tốt nhất giảm dần qua nhiều bước nhỏ và ổn định tại 0.750268 ở thế hệ 392. Ở giai đoạn cuối, các

cá thể tốt nhất đã tập trung quanh vùng nghiệm có chất lượng cao nên các phép biến đổi tiếp theo khó tạo ra giá trị mục tiêu nhỏ hơn đáng kể, khiến đường hội tụ gần như giữ nguyên.

Với bài toán đơn giản như g11, chỉ có hai biến và một ràng buộc $x_2 = x_1^2$, có thể biến đổi trực tiếp ràng buộc để đưa bài toán về tối ưu một biến. Vì vậy, GA chưa chắc mang lại lợi thế so với cách giải trực tiếp dựa trên cấu trúc của bài toán. Điều này cho thấy GA phù hợp hơn với các bài toán có cấu trúc phức tạp hoặc khó giải bằng các phương pháp trực tiếp.

Bài toán g15 được phát biểu như sau:

$$\begin{aligned} \min_{x \in \mathbb{R}^3} \quad & f(x) = 1000 - x_1^2 - 2x_2^2 - x_3^2 - x_1x_2 - x_1x_3 \\ \text{subject to} \quad & x_1^2 + x_2^2 + x_3^2 - 25 = 0 \\ & 8x_1 + 14x_2 + 7x_3 - 56 = 0 \end{aligned} \quad (2.10)$$

với $0 \leq x_i \leq 10$ cho $i = 1, 2, 3$. Nghiệm tối ưu công bố là

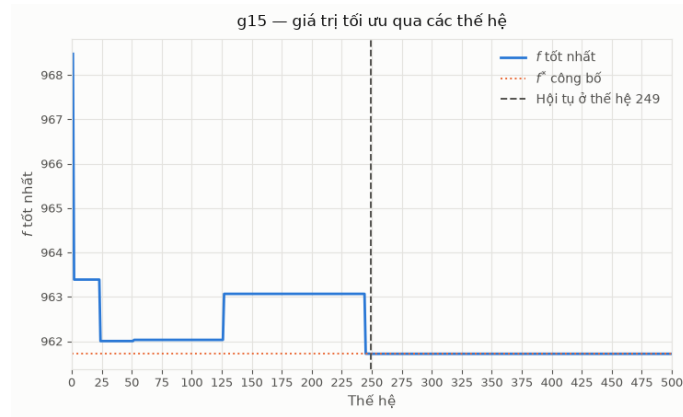
$$x^* = (3.512128; 0.216988; 3.552179), \quad f(x^*) = 961.715022.$$

g15 có ba biến và hai ràng buộc đẳng thức, gồm một ràng buộc phi tuyến và một ràng buộc tuyến tính. Hai ràng buộc đẳng thức của g15 tương ứng với một mặt cầu và một mặt phẳng; giao của chúng là một đường tròn trong không gian ba chiều.

Bảng 2.6: Kết quả trên bài toán g15

f^* công bố	f^* (GA)	Sai lệch (GA)	f^* (GA + tinh chỉnh)	Sai lệch (GA + tinh chỉnh)	Số thế hệ thỏa mãn
961.715022	961.715374	3.52×10^{-4}	961.715308	2.86×10^{-4}	249

Theo Bảng 2.6, GA đạt $f^* = 961.715374$, với sai lệch 3.52×10^{-4} so với nghiệm tối ưu công bố 961.715022. Sau bước tinh chỉnh cục bộ, sai lệch giảm xuống 2.86×10^{-4} , cho thấy tinh chỉnh có cải thiện nhưng mức độ không lớn. Thuật toán đạt tiêu chí thỏa mãn tại thế hệ 249.



Hình 2.10: Đường hội tụ của GA trên bài toán g15

Hình 2.10 cho thấy giá trị tốt nhất biến động rõ rệt tại thời điểm quần thể tìm được cá thể thỏa mãn đồng thời hai ràng buộc. Trước thế hệ 127, giá trị tốt nhất đạt khoảng 962.03; khi xuất hiện cá thể khả thi, giá trị tăng lên 963.07 do quy tắc Deb ưu tiên tính khả thi hơn giá trị hàm mục tiêu. Sau đó, giá trị giảm dần và tiến gần 961.7154 từ khoảng thế hệ 245, rồi ổn định hoàn toàn tại thế hệ 249. Diễn biến này cho thấy phần lớn quá trình cải thiện diễn ra sau khi quần thể đã xác định được vùng khả thi.

Nhận xét và đánh giá kết quả

Bảng 2.7 tổng hợp kết quả trên cả năm bài toán. Mọi nghiệm báo cáo đều khả thi tuyệt đối theo định nghĩa của bộ benchmark, với mức vi phạm bằng 0.

Bảng 2.7: Tổng hợp kết quả GA trên năm bài toán chuẩn CEC 2006

Bài toán	f^* công bố	f^* (GA)	f^* (GA)		Thời gian hội tụ (s)	Số thế hệ thỏa mãn
			+ tính chỉnh	Sai lệch		
g01	-15.000000	-14.999204	-15.000000	1.77×10^{-12}	18.74	499
g06	-6961.813876	-6898.462524	-6961.813876	2.64×10^{-8}	14.06	457
g08	-0.095825	-0.095825	-0.095825	2.78×10^{-17}	13.20	362
g11	0.749900	0.750268	0.750259	3.59×10^{-4}	13.31	392
g15	961.715022	961.715374	961.715308	2.86×10^{-4}	13.82	249

Kết quả cho thấy độ khó của các bài toán ràng buộc đối với GA không chỉ phụ thuộc vào số biến hay độ phức tạp của hàm mục tiêu, mà còn chịu ảnh hưởng đáng kể bởi hình học của miền khả thi và đặc trưng của hàm mục tiêu quanh nghiệm tối ưu. Khi miền khả thi hẹp, như ở g11, g15 hoặc vùng giao của các ràng buộc cong ở g06, lai ghép và đột biến dễ tạo ra các cá thể vi phạm ràng buộc, làm giảm hiệu quả tìm kiếm. Ngược lại, các bài toán có miền khả thi rộng hơn như g08 hoặc cấu trúc hình học thuận lợi như g01 tạo điều kiện để GA duy trì các cá thể khả thi và tiếp cận vùng nghiệm tốt hơn.

g06 thể hiện rõ nhất sự kết hợp của miền khả thi hẹp và hàm mục tiêu biến thiên

mạnh, khiến sai lệch nhỏ về vị trí có thể dẫn đến sai lệch đáng kể về giá trị hàm mục tiêu. Vì vậy, đây cũng là bài toán mà tìm kiếm cục bộ mang lại mức cải thiện lớn nhất, giảm sai lệch từ 63.35 xuống 2.64×10^{-8} . Kết quả này cho thấy GA phù hợp với vai trò xác định vùng chứa nghiệm, trong khi tìm kiếm cục bộ phát huy hiệu quả ở bước tinh chỉnh nghiệm trong vùng lân cận.

Đối với g11 và g15, sai lệch còn lại có thể chủ yếu liên quan đến dung sai $\varepsilon = 10^{-4}$ được sử dụng cho các ràng buộc đẳng thức. Do đó, sai lệch quan sát được không nhất thiết phản ánh trực tiếp khả năng hội tụ chưa đầy đủ của GA.

2.3 Bài toán Người du lịch: GA hoán vị kết hợp Tìm kiếm Cục bộ

Bài toán người du lịch (Traveling Salesman Problem — TSP) được phát biểu như sau: cho n địa điểm và ma trận khoảng cách $D = (d_{ij})_{n \times n}$ giữa mọi cặp địa điểm, tìm một hoán vị $\pi = (\pi_1, \dots, \pi_n)$ của n địa điểm sao cho tổng độ dài lộ trình khép kín đi qua tất cả các địa điểm đúng một lần là nhỏ nhất:

$$\min_{\pi} L(\pi) = \sum_{k=1}^{n-1} d_{\pi_k, \pi_{k+1}} + d_{\pi_n, \pi_1}. \quad (2.11)$$

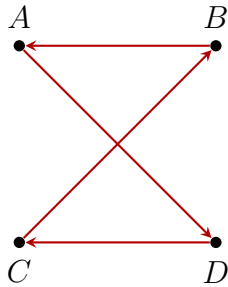
Thực nghiệm trong mục này xét trường hợp TSP đối xứng (Symmetric TSP): khoảng cách giữa hai địa điểm không phụ thuộc chiều di chuyển, tức $d_{ij} = d_{ji}$ với mọi i, j , tương ứng với một đồ thị vô hướng.

Bài toán này được mã hóa dưới dạng hoán vị đã giới thiệu ở Mục 1.2: mỗi cá thể của GA chính là một hoán vị π , dùng lai ghép theo thứ tự và đột biến hoán đổi (Mục 1.3) thay cho các toán tử số thực ở hai mục trước.

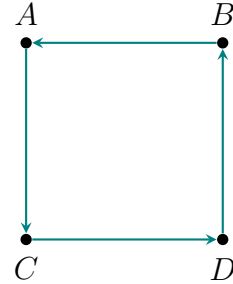
Để đánh giá khách quan trên dữ liệu thực thay vì các ví dụ tự tạo quy mô nhỏ, ba bộ dữ liệu chuẩn từ thư viện benchmark TSPLIB95 [9] (<https://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/tsp/tspindex.html>) được sử dụng: berlin52 ($n = 52$ địa điểm tại Berlin), rd100 ($n = 100$ địa điểm ngẫu nhiên), và ch150 ($n = 150$ địa điểm). Mỗi bộ dữ liệu đều có kèm theo lộ trình tối ưu đã được công bố (<https://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/tsp/TSP-BEST.html>), dùng làm mốc đối chiếu khách quan thay cho phương pháp vét cạn — vốn cần thử $(n - 1)!$ hoán vị và trở nên bất khả thi về mặt tính toán ngay từ n vài chục.

Mục này còn dùng đến hai khái niệm khác: **2-opt** và **memetic algorithm**. 2-opt là một phép tìm kiếm cục bộ riêng cho TSP: xét từng cặp cạnh không kề nhau trong lộ trình, nếu hoán đổi hai cạnh đó — tương đương đảo ngược đoạn đường nằm giữa — làm giảm tổng độ dài thì phép đổi được chấp nhận, lặp lại đến khi không còn cặp cạnh nào cải thiện được nữa; về mặt hình học, mỗi lần đổi tương ứng với việc gỡ bỏ một

chỗ giao nhau giữa hai đoạn đường trên lộ trình. Khi một thuật toán tiến hóa được kết hợp thêm bước tìm kiếm cục bộ như vậy để tinh chỉnh từng cá thể, kết quả được gọi là một memetic algorithm: GA đảm nhiệm khám phá không gian nghiệm trên diện rộng, còn tìm kiếm cục bộ đảm nhiệm hoàn thiện lời giải quanh vùng đã tìm được.



Trước: cạnh A-D và C-B cắt chéo nhau



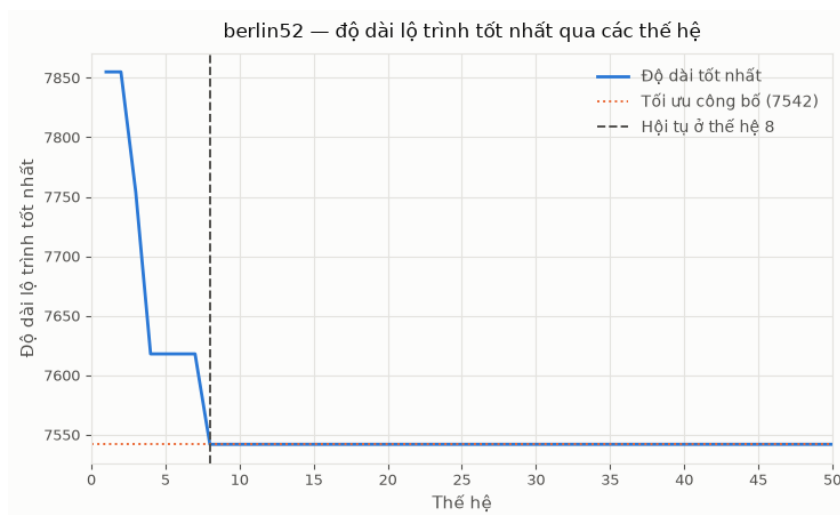
Sau 2-opt: loại bỏ cạnh A-D và C-B, đảo đoạn giữa chúng, tạo tour mới A-C-D-B-A

Hình 2.11: Minh họa một bước 2-opt trên bốn địa điểm: hai cạnh cắt chéo nhau (trái) được thay bằng hai cạnh ngắn hơn, không cắt chéo (phải), bằng cách loại bỏ cạnh A-D và C-B rồi đảo ngược đoạn đường nằm giữa chúng để tạo tour mới A-C-D-B-A.

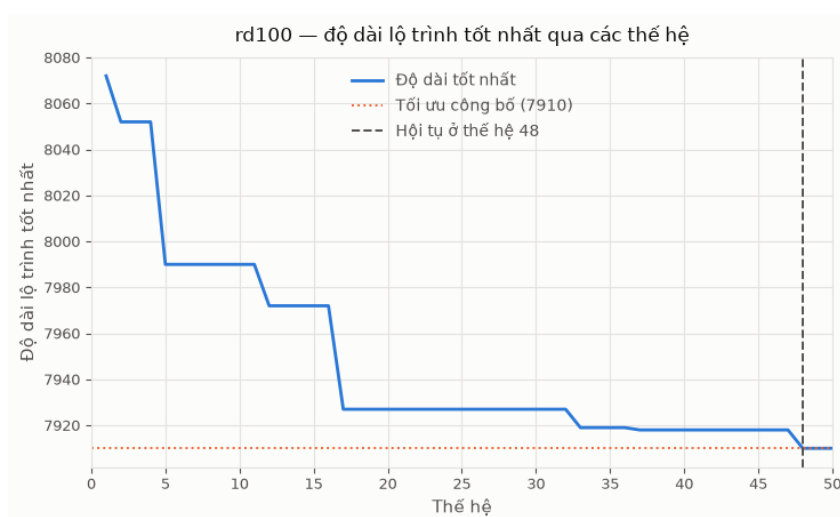
GA hoán vị thuần túy không có cơ chế chủ động sửa các đoạn đường cắt chéo nhau như vậy trong lộ trình, nên hội tụ rất chậm và cách xa nghiệm tối ưu khi số địa điểm tăng lên. Thực nghiệm sử dụng quần thể 1000 cá thể, chạy tối đa 50 thế hệ cho cả ba bộ dữ liệu; kết hợp thêm 2-opt cho một tỉ lệ nhỏ cá thể tốt nhất mỗi thế hệ (khoảng 5% quần thể) khắc phục được hạn chế này: cả ba bộ dữ liệu đều đạt đúng lộ trình tối ưu đã công bố, như trình bày ở Bảng 2.8.

Bảng 2.8: Kết quả GA kết hợp 2-opt trên ba bộ dữ liệu TSPLIB95

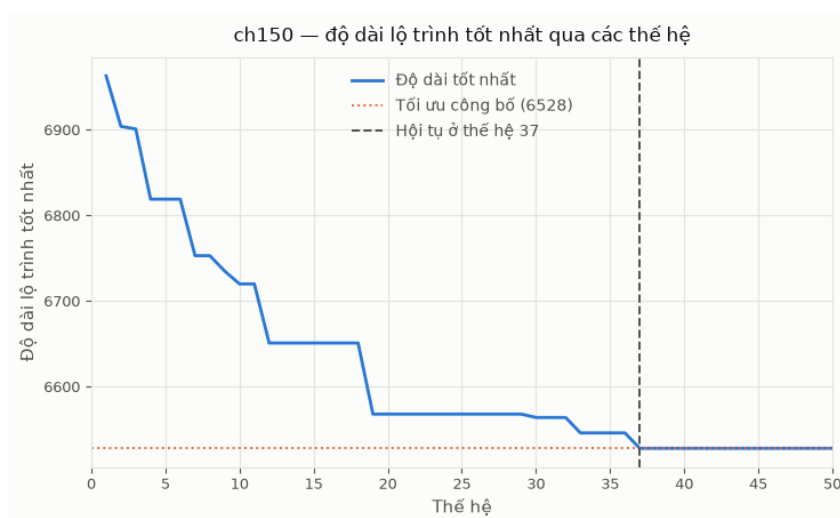
Bộ dữ liệu	Số địa điểm (n)	Nghiệm tối ưu	GA	Đạt được ở thế hệ	Thời gian hội tụ (s)
berlin52	52	7542	7542	8/50	1.83
rd100	100	7910	7910	48/50	7.11
ch150	150	6528	6528	37/50	14.77



Hình 2.12: Đường hội tụ của GA trên bộ dữ liệu berlin52



Hình 2.13: Đường hội tụ của GA trên bộ dữ liệu rd100



Hình 2.14: Đường hội tụ của GA trên bộ dữ liệu ch150

2.4 Hồi quy Symbolic bằng Lập trình Di truyền

Ứng dụng cuối cùng minh họa Lập trình Di truyền (Mục 1.5) cho bài toán hồi quy symbolic trên dữ liệu quan trắc thời tiết thực tế — bộ dữ liệu thời tiết công khai trên Kaggle [10], có thể truy cập tại <https://www.kaggle.com/datasets/alistaairking/weather-long-term-time-series-forecasting>. Mục tiêu dự đoán là nhiệt độ điểm sương T_{dew} (dew point), tính từ hai đặc trưng đo được là nhiệt độ không khí T và độ ẩm tương đối rh . Quan hệ vật lý đúng giữa ba đại lượng này được mô tả bởi công thức Magnus [11]:

$$\gamma(T, rh) = \ln\left(\frac{rh}{100}\right) + \frac{aT}{b+T}, \quad (2.12)$$

$$T_{dew} = \frac{b\gamma}{a-\gamma}, \quad (2.13)$$

với $a = 17.27$ và $b = 237.7$. Đây là một quan hệ phi tuyến có cấu trúc phức tạp, kết hợp logarit và phép chia lồng nhau, hoàn toàn không thể biểu diễn dưới dạng tổ hợp tuyến tính đơn giản của T và rh — điều này khiến bài toán trở thành một phép thử thực tế cho khả năng GP tự khám phá cấu trúc hàm phi tuyến từ dữ liệu mà không cần biết trước dạng công thức (2.13).

Bộ dữ liệu gốc gồm 52 696 dòng quan trắc, mỗi dòng ứng với một mốc thời gian đo cách nhau khoảng 10 phút trong suốt năm 2020, với tổng cộng 21 đặc trưng khí tượng (áp suất, nhiệt độ, độ ẩm, tốc độ gió, bức xạ mặt trời...). Trong số đó, chỉ 4 cột được giữ lại để phục vụ bài toán GP.

Bảng 2.9: Năm dòng đầu tiên của dữ liệu thời tiết sau khi lọc còn 4 cột

date	T	Tdew	rh
2020-01-01 00:10:00	0.71	−1.33	86.1
2020-01-01 00:20:00	0.75	−1.44	85.2
2020-01-01 00:30:00	0.73	−1.48	85.1
2020-01-01 00:40:00	0.37	−1.64	86.3
2020-01-01 00:50:00	0.33	−1.50	87.4

Ý nghĩa của từng cột như sau: **date** là thời điểm đo, chỉ dùng để tham chiếu chứ không đưa vào làm đặc trưng số cho GP; **T** là nhiệt độ không khí đo được (°C); **rh** là độ ẩm tương đối (%), tức tỉ lệ giữa lượng hơi nước hiện có trong không khí so với lượng hơi nước tối đa mà không khí có thể chứa ở cùng nhiệt độ; và **Tdew** là nhiệt độ điểm sương (°C) — nhiệt độ mà tại đó không khí đạt trạng thái bão hòa hơi nước và bắt đầu ngưng tụ thành sương, chính là đại lượng mục tiêu cần dự đoán. Như vậy, T và rh đóng vai trò đặc trưng đầu vào, còn T_{dew} là nhãn.

GP được triển khai với tập toán tử và bộ tham số như sau:

- Tập toán tử: $\{+, \times, \div, \log\}$ (phép chia được bảo vệ như đã trình bày ở Mục 1.5);
- Quần thể 10 000 cá thể, chạy qua 500 thế hệ, độ sâu cây tối đa 6;
- Tỷ lệ lai ghép 0.8, tỷ lệ đột biến 0.2 (chọn ngẫu nhiên đều giữa ba loại đột biến đã trình bày ở Mục 1.5: thay thế cây con, đột biến điểm, và đột biến rút gọn), kích thước elitism 200;
- Hệ số phạt kích thước cây $\lambda = 0.01$.

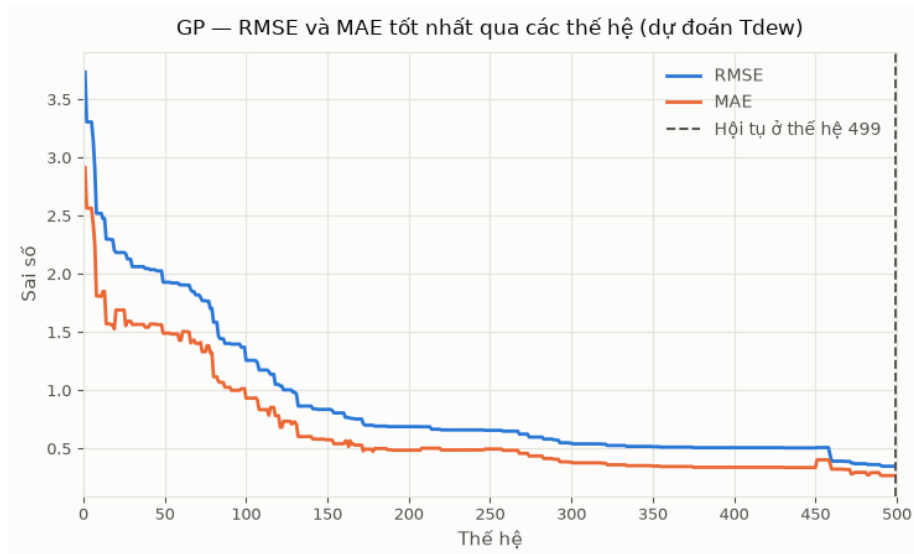
Biểu thức tốt nhất GP tìm được, đạt được ở thế hệ 499/500, là:

$$T_{dew} = \log\left(\log\left(\frac{0.325}{\log(rh)}\right)\right) \times \left[\frac{-1.46}{((-0.121) \cdot rh) \cdot ((-0.121) \cdot 0.161)} + \left((-0.489) \cdot (-0.121) \cdot rh + T\right) \right], \quad (2.14)$$

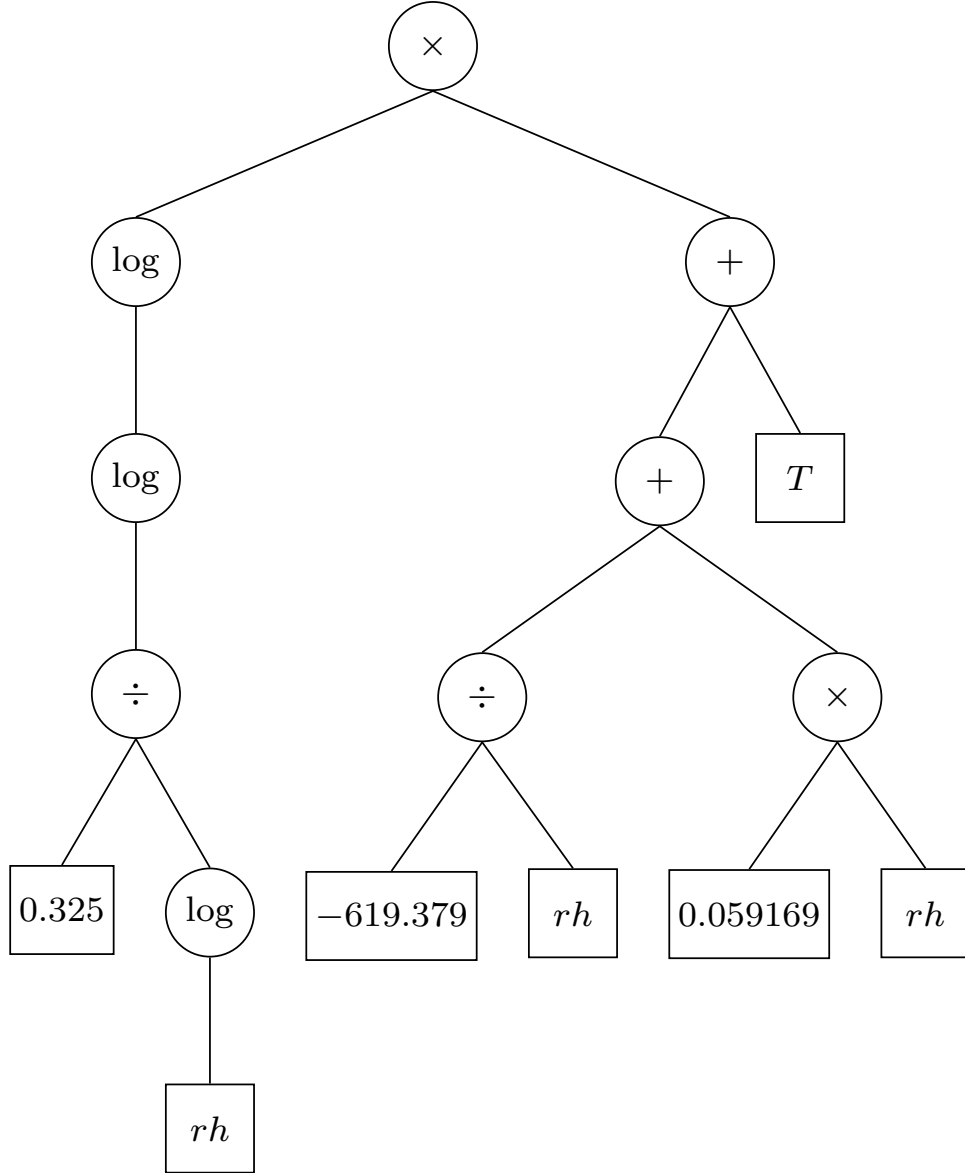
gồm 24 nút và có độ sâu 6 — trùng khớp với giới hạn độ sâu tối đa đã đặt. Rút gọn các tích số giữa hai hằng số ở Phương trình (2.14) về một hằng số duy nhất, biểu thức trên có thể viết lại gọn hơn dưới dạng:

$$T_{dew} \approx \log\left(\log\left(\frac{0.325}{\log(rh)}\right)\right) \times \left(\frac{-619.379}{rh} + 0.059169 rh + T\right). \quad (2.15)$$

Trên tập kiểm tra, biểu thức (2.14) (tương đương Phương trình (2.15)) đạt MAE = 0.2659, RMSE = 0.3439, và $R^2 = 0.9967$, tức giải thích được hơn 99% phương sai của T_{dew} chỉ từ hai đặc trưng đầu vào. Hình 2.15 minh họa quá trình hội tụ của cả RMSE lẫn MAE qua 500 thế hệ, còn Hình 2.16 trình bày trực quan cấu trúc cây của biểu thức tìm được.



Hình 2.15: Đường hội tụ của RMSE và MAE qua các thế hệ khi GP dự đoán T_{dew}



Hình 2.16: Cây biểu thức GP tìm được cho T_{dew} (đã rút gọn theo Phương trình (2.15))

Kết quả thực nghiệm cho thấy Symbolic Regression bằng Genetic Programming (GP) có khả năng học và tái hiện tốt mối quan hệ phi tuyến giữa nhiệt độ T , độ ẩm tương đối rh và nhiệt độ điểm sương T_{dew} . Trên tập kiểm tra, biểu thức tìm được đạt $MAE = 0.2659$, $RMSE = 0.3439$ và $R^2 = 0.9967$, cho thấy mô hình có độ chính xác rất cao dù chỉ sử dụng hai biến đầu vào.

Mặc dù biểu thức GP chưa khôi phục hoàn toàn công thức Magnus, cấu trúc của biểu thức vẫn thể hiện rõ những đặc trưng quan trọng của quan hệ vật lý, đặc biệt là sự xuất hiện lặp lại của $\log(rh)$ và các phép chia lồng nhau. Điều này cho thấy GP không chỉ tìm được một hàm có khả năng dự đoán tốt mà còn có thể tự khám phá cấu trúc toán học phù hợp với hiện tượng thực tế.

Sự khác biệt so với công thức Magnus có thể một phần xuất phát từ giới hạn biểu diễn của engine, khi các hằng số tại nút lá chỉ nằm trong khoảng $[-2, 2]$, trong khi

công thức Magnus sử dụng các hằng số lớn hơn đáng kể. Do đó, việc biểu diễn trực tiếp các hằng số này đòi hỏi phải kết hợp nhiều nút, làm tăng độ phức tạp của cây. Nhìn chung, thí nghiệm cho thấy GA/GP là một hướng tiếp cận khả thi cho Symbolic Regression, vừa đạt độ chính xác cao vừa cung cấp một biểu thức toán học có khả năng diễn giải, thay vì chỉ đưa ra một mô hình dự đoán dạng hộp đen.

Kết luận

Tóm tắt

Tiểu luận đã trình bày cơ sở lý thuyết của Thuật toán Di truyền (GA) và mở rộng của nó là Lập trình Di truyền (GP), sau đó minh họa cả hai phương pháp qua bốn thực nghiệm cụ thể. Chương 1 xây dựng khung lý thuyết theo trình tự: tổng quan bài toán tối ưu và họ thuật toán tiến hóa; cơ sở khoa học của GA với ba cách mã hóa cá thể (nhị phân, số thực, hoán vị); các thành phần cốt lõi của một vòng lặp tiến hóa (hàm thích nghi, chọn lọc, lai ghép, đột biến, elitism); quy trình thuật toán và điều kiện dừng; và cuối cùng là GP với mã hóa dạng cây biểu thức, mở rộng GA sang bài toán tự động tổng hợp chương trình thay vì chỉ tối ưu tham số.

Trên nền tảng đó, Chương 2 áp dụng GA cho ba nhóm bài toán tối ưu số — không ràng buộc trên năm hàm benchmark kinh điển, có ràng buộc trên năm bài toán chuẩn CEC 2006, và bài toán người du lịch kết hợp 2-opt local search theo mô hình memetic algorithm — đối chiếu trực tiếp với các mốc nghiệm tối ưu đã công bố. Ở cả ba nhóm, GA (khi cần thiết được hỗ trợ thêm bởi tìm kiếm cục bộ) đều đạt hoặc tiệm cận nghiệm tối ưu đã biết. Riêng ứng dụng thứ tư dùng GP cho bài toán hồi quy symbolic, dự đoán nhiệt độ điểm sương từ nhiệt độ không khí và độ ẩm tương đối trên dữ liệu thời tiết thực tế, đạt $R^2 = 0.9967$ trên tập kiểm tra và tìm ra một biểu thức tường minh phản ánh đúng cấu trúc phi tuyến (logarit, phép chia lồng nhau) của quan hệ vật lý thật (công thức Magnus), thay vì chỉ đưa ra một mô hình dự đoán dạng hộp đen.

Hạn chế

Bên cạnh các kết quả đạt được, tiểu luận còn một số hạn chế cần được nhìn nhận rõ. Thứ nhất, tính linh hoạt của GA/GP — không đặt giả định trước về cấu trúc bài toán — đi kèm với hiệu quả thấp hơn các phương pháp chuyên dụng khi bài toán có sẵn cấu trúc khai thác được: GA thuần túy cần bổ sung tìm kiếm cục bộ mới tiếp cận được nghiệm nằm sát biên miền khả thi ở các bài toán CEC 2006, và cần kết hợp 2-opt mới hội tụ đủ nhanh, đủ gần nghiệm tối ưu ở bài toán người du lịch quy mô lớn. Thứ hai, engine GP tự cài đặt trong tiểu luận giới hạn hằng số tại nút lá trong khoảng $[-2, 2]$, khiến việc tổng hợp chính xác các hằng số lớn (như a, b trong công thức Magnus) đòi hỏi tổ hợp nhiều nút, tiêu tốn ngân sách độ sâu vốn cần dùng cho cấu trúc chính của biểu thức — đây là nguyên nhân chính khiến GP chưa khôi phục

được chính xác tuyệt đối công thức Magnus. Thứ ba, các thực nghiệm trong tiểu luận đều chỉ chạy với một giá trị seed cố định cho mỗi cấu hình tham số, chưa lặp lại nhiều lần với các seed khác nhau để đánh giá phương sai của kết quả; quy mô bài toán thử nghiệm (số chiều trong tối ưu ràng buộc, số địa điểm trong TSP, số đặc trưng đầu vào trong hồi quy symbolic) cũng còn tương đối nhỏ so với các bài toán thực tế phức tạp hơn.

Hướng phát triển

Từ những hạn chế trên, tiểu luận đề xuất một số hướng phát triển cho các nghiên cứu tiếp theo. Một là, thực hiện nhiều lần chạy độc lập với các seed khác nhau cho mỗi cấu hình thực nghiệm, báo cáo kết quả dưới dạng giá trị trung bình và độ lệch chuẩn, nhằm đánh giá đầy đủ hơn độ ổn định của thuật toán thay vì chỉ dựa trên một lần chạy duy nhất. Hai là, mở rộng quy mô bài toán thử nghiệm — số chiều lớn hơn cho tối ưu có ràng buộc, số địa điểm lớn hơn cho bài toán người du lịch, số đặc trưng đầu vào nhiều hơn cho hồi quy symbolic — để đánh giá khả năng mở rộng (scalability) của các thuật toán đã cài đặt. Ba là, đối với GP, có thể bổ sung một bước tối ưu cục bộ riêng cho các hằng số tại nút lá (ví dụ bằng gradient descent hoặc least-squares sau khi cấu trúc cây đã cố định), thay vì chỉ dựa vào đột biến ngẫu nhiên để tinh chỉnh hằng số — đây là một kỹ thuật đã được chứng minh hiệu quả trong nhiều hệ thống GP hiện đại và có thể giúp khắc phục trực tiếp hạn chế về khoảng giá trị hằng số nêu ở phần Hạn chế. Bốn là, có thể thử nghiệm các cơ chế tham số thích ứng (self-adaptive), trong đó tỉ lệ lai ghép, đột biến, hoặc kích thước quần thể tự điều chỉnh theo tiến trình hội tụ thay vì cố định trong suốt quá trình chạy. Cuối cùng, ý tưởng kết hợp GA/GP với tìm kiếm cục bộ chuyên dụng — đã chứng minh hiệu quả rõ rệt ở bài toán người du lịch trong tiểu luận này — có thể được mở rộng sang các nhóm bài toán còn lại, chẳng hạn tích hợp một bước tối ưu cục bộ dựa trên gradient cho các bài toán tối ưu có ràng buộc khi hàm mục tiêu và ràng buộc khả vi.

Tài liệu tham khảo

1. Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor. (2nd Edition, MIT Press, 1992.)
2. Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
3. Deb, K. (2000). An efficient constraint handling method for genetic algorithms. *Computer Methods in Applied Mechanics and Engineering*, 186(2–4), 311–338.
4. Takahama, T., & Sakai, S. (2010, July). Constrained optimization by the ε constrained differential evolution with an archive and gradient-based mutation. In *IEEE Congress on Evolutionary Computation* (pp. 1–9). IEEE.
5. De Jong, K. A. (1975). *An Analysis of the Behavior of a Class of Genetic Adaptive Systems*. University of Michigan.
6. Koza, J. R. (1994). Genetic programming as a means for programming computers by natural selection. *Statistics and Computing*, 4(2), 87–112.
7. Surjanovic, S., & Bingham, D. (2013). Virtual Library of Simulation Experiments: Test Functions and Datasets. Simon Fraser University. <https://www.sfu.ca/~ssurjano/optimization.html>
8. Liang, J. J., Runarsson, T. P., Mezura-Montes, E., Clerc, M., Suganthan, P. N., Coello, C. C., & Deb, K. (2006). Problem definitions and evaluation criteria for the CEC 2006 special session on constrained real-parameter optimization. *Journal of Applied Mechanics*, 41(8), 8–31.
9. Reinelt, G. (1991). TSPLIB — A traveling salesman problem library. *ORSA Journal on Computing*, 3(4), 376–384.
Danh sách bộ dữ liệu: <https://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/tsp/tspindex.html>
Lộ trình tối ưu đã công bố: <https://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/tsp/TSP-BEST.html>

10. King, A. Weather Long-term Time-series Forecasting [Data set]. *Kaggle*. <https://www.kaggle.com/datasets/alistairking/weather-long-term-time-series-forecasting>
11. Wikipedia contributors. Dew point. *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Dew_point